Wisdom via Multiple Perspectives: A Multi-granularity Clusters Fusion Approach for Fault Diagnosis with Noisy Labels

Journal:	*Transactions on Mechatronics*
Manuscript ID	TMECH-11-2024-19463.R1
Manuscript Type:	Regular paper (S1)
Date Submitted by the Author:	23-Feb-2025
Complete List of Authors:	Dunkin, Fir; Southeast University - Sipailou Campus, School of Automation Li, Xinde; Southeast University, School of Automation Wu, Guoliang; Southeast University, School of Automation Hu, Chuanfei; Southeast University, Yu, Le; Southeast University, School of Automation Lu, Xiaoyan; Southeast University Ge, Shuzhi; National University of Singapore, Department of Electrical and Computer Engineering
Keywords:	Fault diagnosis & prognosis < Manufacturing, AI and machine learning < Intelligent control, Signal and image processing < Intelligent control
Are any of authors IEEE Member?:	Yes
Are any of authors ASME Member?:	No

Wisdom via Multiple Perspectives: A Multi-granularity Clusters Fusion Approach for Fault Diagnosis with Noisy Labels

Fir Dunkin, Xinde Li*, *Senior Member IEEE*, Guoliang Wu, Chuanfei Hu, Le Yu, Xiaoyan Lu, and Shuzhi Sam Ge, *Fellow IEEE*

***Abstract*—Deep Neural Networks (DNNs) have shown excellent performance in fault diagnosis, but this heavily relies on training datasets with high-quality labels. However, both manual annotation and automatic annotators inevitably introduce noisy labels into dataset, which mislead the model during training and negatively affect generalization, especially in strong noisy environments. Therefore, this article proposes a multi-granularity label construction approach via Multi-granularity Cluster Fusion (*MgCF*), aiming to more efficiently suppress the misleading effect of noisy labels. MgCF first constructs a granular cluster for each sample in the latent feature space based on the estimation of the noise intensity of training dataset, and estimates the membership relationship of the sample to each category based on the distribution of labels in the cluster. Then, by fusing the membership relationship with the original observation label, a corrected multi-granularity label is obtained. Finally, the fusion label replaces the original label to complete the training process. A series of experimental results and theoretical analysis confirm MgCF's effectiveness, offering a robust training strategy for intelligent fault diagnosis even with low-quality, cost-effective datasets. MgCF has the potential to expand the practical application of such datasets, advancing the development of intelligent mechatronic systems.**

***Index Terms*—Learning with noisy labels, Feature fusion, Fuzzy inference, Granular computing, Time series classification.**

I. Introduction

MODERN mechanical systems, including shield tunneling machines, aviation engines, and high-speed trains, require high levels of reliability and safety for long-term operation in harsh environments [1]. With advancements in deep learning and transfer learning, intelligent diagnostic methods such as Deep Adversarial Capsule Network (DACN) [2] and Knowledge Embedded Autoencoder Network (KEA) [3] have been developed, demonstrating the potential for handling compound fault diagnosis and generalizing across domains even under complex and variable working conditions.

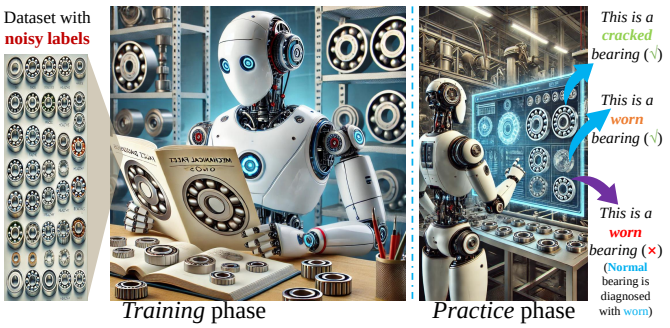


Fig. 1. The misdirecting effect of noisy labels on intelligent diagnostic models: When there are noisy labels in the training dataset, the model overfits the incorrect label information, resulting in the same erroneous decision in practice.

However, due to their core components, e.g. bearings, which are prone to fatigue degradation, these systems may experience functional failures, and if these failures cannot be diagnosed in a timely and accurate manner, it will lead to significant economic losses and even casualties [4]. Therefore, fault diagnosis plays an important role in the safety maintenance of complex mechanical systems.

With the rapid development of artificial intelligence technology, data-driven intelligent diagnostic methods have been widely studied and applied in academia and industry [5]–[8]. Advanced deep learning methods, such as WavCapsNet [9] and DACN [2], leverage neural networks' powerful feature extraction capability to efficiently decouple and diagnose compound faults, even under unseen or multidomain scenarios. Comprehensive studies [10], including surveys on deep transfer learning for fault diagnosis, underscore the significance of these advancements in addressing real-world industrial challenges.

It is well recognized that the performance of DNNs is heavily dependent on the size of the dataset and the quality of annotations [11], i.e. most current intelligent diagnostic methods operate under the assumption that the labels are entirely accurate. However, producing high-quality annotations is an exceptionally costly task [12]. To reduce labeling costs, the crowdsourcing [13] and semi-automated [14] annotation methods have gained attention. Unfortunately, these methods

This work was supported in part by the National Natural Science Foundation of China under Grant 62233003 and 62073072, the Key Projects of Key R&D Program of Jiangsu Province under Grant BE2020006 and BE2020006-1, and the Shenzhen Science and Technology Program under Grant JCYJ20210324132202005 and JCYJ20220818101206014 (*Corresponding author: *Xinde Li*).

Fir Dunkin, Xinde Li, Guoliang Wu, Chuanfei Hu, Le Yu, and Xiaoyan Lu are with Key Laboratory of Measurement and Control of CSE, School of Automation, Southeast University, Nanjing, China (Emails: dunkin2fir@ieee.org; xindeli@seu.edu.cn; 230208677@seu.edu.cn; chuanfei_hu@ieee.org; leyu@seu.edu.cn; 15850500060@163.com).

Xinde Li also is with the Southeast University Shenzhen Research Institute, Shenzhen, China.

Shuzhi Sam Ge is with the Social Robotics Laboratory, Department of Electrical and Computer Engineering, Interactive Digital Media Institute, National University of Singapore, Singapore (Email: samge@nus.edu.sg).

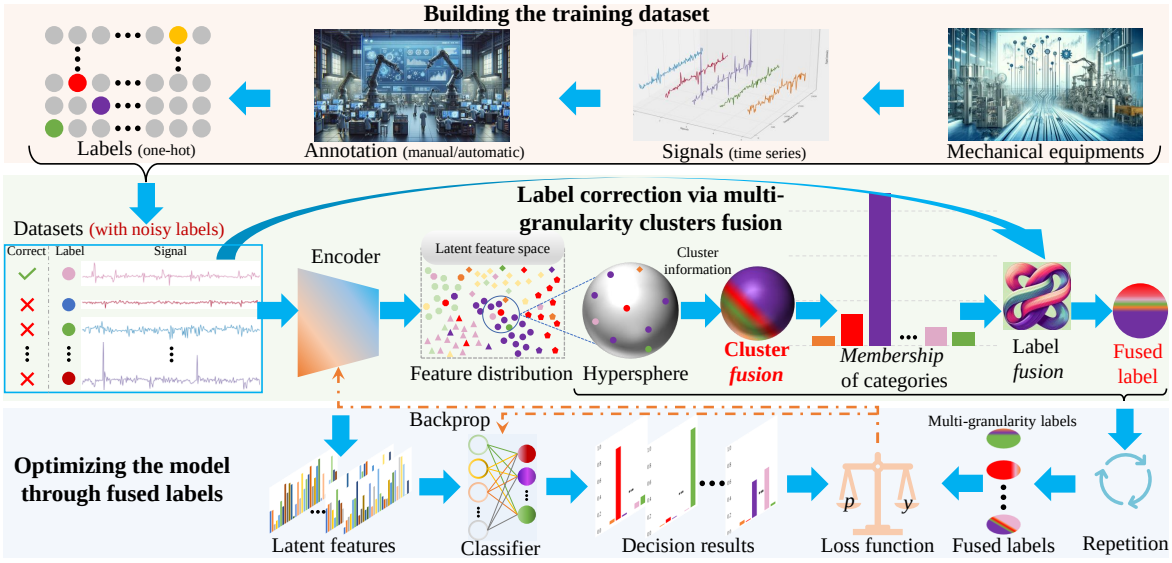


Fig. 2. The overview of MgCF: For datasets with noisy labels, MgCF starts by using a feature encoder (the backbone of deep diagnostic model) to map the input sample signals into a high-dimensional feature space. Then, MgCF dynamically constructs feature clusters with specific granularity, based on the estimated noise intensity in the dataset, for each feature representation. Next, MgCF integrates the information of all observation labels corresponding to the features within each cluster on a hypersphere to estimate the category membership of current sample. Subsequently, the category membership is defuzzified to generate a predicted label with a category membership degree, and this predicted label, combined with the original observation label, forms multi-granularity labels for self-guided learning, thereby completing the supervised training of the diagnostic model.

inevitably introduce noise in the labels of training dataset [15], where the observed labels annotated are inconsistent with the actual true labels.

Once a dataset is laden with a significant amount of noisy labels, DNNs tend to overfit to these inaccuracies, diminishing their generalization performance (Fig. 1), which reduction in performance can lead to inadequate accuracy for automated diagnostic needs [16]. Consequently, training diagnostic models with noisy labels presents a critical challenge that must be overcome to enhance the intelligence level of maintenance for complex machinery systems.

To enhance the performance of diagnostic models, several methods for dealing with noisy labeled data have recently emerged, broadly categorized into architecture modification [17], loss adjustment [18], and sample separation [7]. In architecture modification [19], a common approach involves integrating a noise adaptation layer before the decision-making layer of the model, utilizing an estimated noise transition matrix to alter the model structure to mitigate the misguidance from noisy labels. However, the accuracy of this estimation may degrade as the number of fault types increases [20], rendering it less effective in increasingly complex modern mechanical or industrial systems. Regarding loss adjustment methods [21], they work by altering the loss values across all training samples to minimize the negative impact of noisy labels. However, this approach may incur cumulative errors, particularly when dealing with a large number of classes or high noise levels [12]. Lastly, sample separation methods [22] establish a confidence function for each label to filter the dataset, which tend to select labels that are highly likely to be non noisy for training, e.g. filtering out samples with incorrect labels via the small loss criterion [23], assuming that the loss of clean samples is less than that of noisy samples.

Although sample separation methods generally yield competitive results, they still have a critical limitation: information loss. Traditional sample separation methods select clean samples from noisy datasets for training and discard those with noisy labels [24], leading to the exclusion of potentially valuable data from the training process [12]. While some semi-supervised techniques incorporate filtered noisy samples into self-supervised learning frameworks, preserving their input information [20], these methods introduce complex training procedures that not only increase time consumption but also discard the label information of the "noisy" samples. Furthermore, most current methods for learning with noisy labels essentially rely on their own prediction or feature extraction results to guide learning within a noisy environment [11], that is, their training is a self-guided learning process. During training, models inevitably hesitates about the membership degrees of certain samples at the distribution boundary, leading to inaccurate predictions and incorrect sample separation, which result in accumulated errors in subsequent self-guided learning process, diminishing the generalization ability and feature extraction capability of the diagnostic models.

These significant challenges have inspired the research motivation of this work:

> Despite unknown noise ratios, *how to utilize the label information from* ***"noisy" samples****, to suppress confirmation bias in self-guided learning* , thereby maximizing the model's generalization capacity.

In this article, with the aim of enhancing the robustness of the training by integrating both reliable and uncertain labels, our proposed **M**ulti-**g**ranularity **C**luster **F**usion (*MgCF*, see Fig. 2) innovatively incorporates a two-layer label confidence modeling mechanism, creating a more adaptive noise-tolerant learning paradigm compared to single-granularity label correction approaches.

According to the observations in Refs. [25]–[27], during training, the loss of clean samples are typically lower than those of samples with noisy labels, which indicates that the sample distribution information in the latent feature space

constructed by the DNN has some degree of class distinction capability. Based on this phenomenon, we hypothesize that samples with similar features in the latent feature space are likely to have the same labels. Therefore, by dynamically constructing feature clusters, MgCF estimates the category membership based on the consistency and similarity of features within the cluster, thus introducing a more resilient label correction strategy.

However, it is evident that the inferred predicted labels cannot be entirely accurate. To prevent overfitting caused by error accumulation during training, MgCF further constructs multi-granularity labels by integrating the original observed labels with the inferred predicted labels. Specifically, when the predicted label matches the observed label, the fine-grained observed label is used for training. When there is a mismatch, a coarse-grained label that retains both class information is used, with the importance of the two classes adjusted according to their category membership degree.

Our contributions are summarized as follows:

1) **Theoretical contribution**: We explored a paradigm of cluster information fusion that effectively integrates feature distribution within clusters to infer the category membership of cluster center samples, which expands the theoretical framework for multi-granularity information fusion via introducing a novel approach for decision attribute acquisition.
2) **Methodological contribution**: By using fused multi-granularity labels instead of original fine-grained labels, we reduce the cumulative error in the self-guided process of learning with noisy labels, which offers a more robust training method for practitioners in related fields, not limited to fault diagnosis.
3) **Empirical contribution**: Extensive experimental results demonstrate the effectiveness of the proposed MgCF in various noisy environments. Notably, it significantly enhances the model's tolerance to noisy labels in high-noise settings, which makes it feasible to use automatically annotated datasets with lower annotation costs in a wider range of fault diagnosis tasks.

II. Preliminary

A. Problem formulation

1) Fault diagnosis with noisy labels: Fault diagnosis task with noisy labels can be framed as a multi-class problem within the context of noisy label learning [28]. Given an industrial time series dataset $\mathcal{D} = \{\mathcal{X}, \hat{\mathcal{Y}}\} = \{(x^{(i)}, \hat{y}^{(i)})\}_{i=1}^{N}$, where N denotes the number of training samples, $\hat{y}^{(i)} \in [1, K]$ represents the observed label for $x^{(i)}$, and K denotes the total number of categories. Also $x^{(i)} \in \mathbb{R}^{C\times L}$ refers to the time series input signal comprising L consecutive sampling points across C channels. Let $y^{(i)}$ be the true label for the input signal $x^{(i)}$. In the training dataset $\mathcal{D}$, labels $\hat{y}^{(i)}$ that satisfy $\hat{y}^{(i)} = y^{(i)}$ are considered clean labels, whereas those that do not are referred to as noisy labels. Our work focuses on training diagnostic models under both symmetric (uniform) and asymmetric (class-dependency) noise conditions.

In symmetric label noise, each class label has an equal probability of being incorrectly labeled as another class, with a uniform error probability denoted by noise rate η, and the label noise transition matrix is shown in Eq. (1).

$$P(y = k \mid \hat{y} = l) = \begin{cases} 1-\eta, & \text{if } k = l \\ \frac{\eta}{K-1}, & \text{otherwise} \end{cases} \tag{1}$$

In asymmetric label noise, the incorrect labeling depends on the similarity or specific relationships between classes. That is, the transition probabilities η are not uniformly distributed but vary across classes. The noise transition probability $P(\hat{y} = k \mid y = l)$ between classes k and l can be described by an asymmetric transition matrix Eq. (2).

$$P(y = k \mid \hat{y} = l) = \begin{cases} 1-\eta, & \text{if } k = l \\ \eta_{kl}, & \text{otherwise} \end{cases} \tag{2}$$

Where, η_{kl} represents the probability of class k being mislabeled as class l, with $\sum_{k\neq l} \eta_{kl} = \eta$.

Thereupon, the fault diagnosis problem can be formulated as follows: for an intelligent diagnostic model $f(\Theta_b, \Theta_c; \cdot)$, where Θ_b and Θ_c denote the parameters of the model backbone and classifier respectively, we aim to employ a training strategy that minimizes the discrepancy between the diagnostic results $\mathcal{P} = f(\Theta_b, \Theta_c; \mathcal{X})$ and the true fault conditions $\mathcal{Y}$, i.e. $\underset{\Theta_b,\Theta_c}{\operatorname{argmin}}\{\mathbb{E}(\mid \mathcal{P} - \mathcal{Y} \mid)\}$.

2) Basic definitions :

Definition 1: (**Minimizing the risk) [29]** *For a given DNN f and a loss function ℓ during training, let p denote the prediction results given by the classifier and $\hat{y}$ is the observation label of x given by dataset $\mathcal{D}$. The risk of the classifier f is defined as $R(f) = \mathbb{E}_{\mathcal{D}}[\ell(p, \hat{y})]$, where $\mathbb{E}$ represents the expectation operator, then when obtaining the global optimal classifier f_g, the minimum risk of classifier f can be obtained, which is $R(f_g) = \min_{f\in\mathcal{F}} R(f)$ (i.e. $\forall f \in \mathcal{F}, \quad R(f_g) \leq R(f)$).*

Definition 2: (**Noisy tolerance**) [21] *For a given DNN f and a training dataset $\mathcal{D}$ with noisy labels by a transition matrix governed by the probability η, a training paradigm $\Phi(\mathcal{D}; \cdot)$ is considered noise-tolerant or robust if it ensures that the minimized risk $R^{\eta}(f_g)$ of the optimal DNN $f_g = \Phi(\mathcal{D}; f)$ trained under this paradigm is equal to the minimized risk $R(f_g)$ on the dataset without noisy labels. In other words, the training paradigm $\Phi(\mathcal{D}; \cdot)$ is noisy tolerance if $R^{\eta}(f_g) = R(f_g)$.*

Definition 3: (**Multi-granularity labels**) *A one-hot label $y^f = [y_1, y_2, \ldots, y_K]$ is defined such that $y_k = 1$ if the sample belongs to the k-th class, and $y_i = 0$ for $i \neq k$ in a K-class problem. This label y^f is referred to as a fine-grained label, indicating the sample is classified exclusively into a single class. Similarly, a label $y^c = [y_1, y_2, \ldots, y_K]$ is defined where $0 \leq y_i \leq 1$ for each i, and $\sum_{i=1}^{K} \mathbb{1}(y_i \neq 0) = c$, indicating that the sample is potentially classified into c out of K classes without specifying the exact class within these c classes. This label y^c is referred to as a coarse-grained label. For a dataset $\mathcal{D}$, if the labels include both fine-grained labels indicating specific classes and coarse-grained labels*

indicating uncertainty among multiple classes, the labels of this dataset is said to have multi-granularity labels.

B. Related works on learning with noisy labels

Due to their excellent non-linear fitting capabilities, DNNs with sufficient capacity tend to overfit even moderately sized clean datasets [11]. Consequently, they are prone to overfitting noisy label samples, resulting in poor generalization to unseen data and significantly reduced accuracy in practical applications. To address overfitting, various methods [30]–[35] have been proposed for learning with noisy labels.

Early studies [30], [31] utilized a small set of clean samples to re-label noisy samples based on modeling from this clean subset. Subsequent researches [32]–[34] focused on distinguishing clean samples from a given noisy dataset, reconstructing a dataset from these clean samples while ignoring the noisy ones, and the most common criterion for selecting clean samples is loss minimization [32]. Recently, feature representation-based noise filtering strategies, such as TopoFilter [33] based on spatial topological patterns and SSR [34] using KNN classifiers, have emerged. However, in high-noise environments, discarding too many "noisy" samples can lead to underfitting, as the remaining "clean" samples may not provide sufficient information to meet the model's training requirements, resulting in poor convergence and lower accuracy.

To address this, some approaches [22], [28] have introduced unsupervised strategies like contrastive learning [20] during training. These semi-supervised methods aim to utilize the input information of "noisy" samples to avoid underfitting due to insufficient sample volume, and have been shown to improve the classification accuracy of neural networks trained on noisy datasets, despite making the training process more complex and time-consuming [12].

Simultaneously, other researchers have focused on label correction, often using model predictions to correct sample labels. For instance, Jo-SRC [35] employs the mean-teacher model to generate more reliable pseudo-label distributions for training.

However, both label correction and sample separation fundamentally rely on the predictions of DNN, making them iterative self-guiding processes [11] that discard "noisy" label information. Neither strategy can guarantee perfect modeling of the separation task at the start of training (i.e., accurately correcting noisy labels to true labels or perfectly distinguishing noisy from clean labels), which introduces the risk of accumulating errors during training, especially in high-noise environments.

With the intention of mitigating the cumulative error risk during the self-guiding process, MgCF proposed a multi-granularity label training approach, aiming to maximize the utilization of all information in the original dataset, even if it may be noisy labels, to improve the model's tolerance (robustness) to noisy labels in high-noise environments. That is to say, the **key difference** between MgCF and current approaches for learning with noisy labels is that *multi-granularity labels* with two layers of granularity, coarse and fine, are used instead of fine-grained labels that only annotate certain category information.

III. Methodology

A. Design rationale of MgCF

According to Refs [7], [11], well-generalizing DNNs tend to form multiple distinct feature clusters in their latent feature space that are easily separable by class. Based on this, we hypothesize that if a model generalizes well, the similarity between two features in the feature space, mapped by the model's backbone, can approximate the probability that the two samples have the same label (Assumption 1). Furthermore, we assume that if a batch of samples forms a feature cluster in the feature space, the label of the sample corresponding to the cluster center is likely to be the same as the majority of the samples in the cluster. Thus, MgCF proposes a cluster fusion method that calculates the category membership of the cluster center sample based on the similarity between other features in the cluster and the cluster center feature, as well as their corresponding observed labels, thereby generating a pseudo-label for the cluster center sample based on the predictions of current model.

Assumption 1: (**Label consistency**) *For* $\forall x^{(i)}, x^{(j)} \in \mathcal{D}$*, if* $f(\Theta_b, \Theta_c; \cdot)$ *generalizes well, then* $Sim(z^{(i)}, z^{(j)}) \propto p(y^{(i)} = y^{(j)})$*. Where* $\mathcal{Z} = f(\Theta_b; \mathcal{X})$*, and* $Sim(z^{(i)}, z^{(j)})$ *represents the similarity between* $z^{(i)}$ *and* $z^{(j)}$*.*

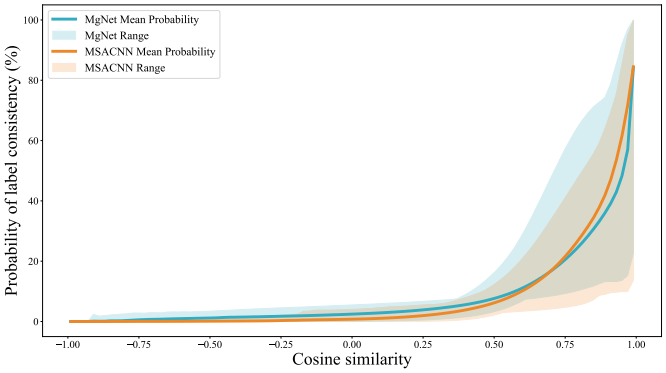


Fig. 3. The relationship between the probability of any two samples having the same label and their feature similarity.

Moreover, considering that DNNs cannot perfectly predict the true class information of all samples during training, directly replacing the original observed labels with pseudo-labels obtained from cluster fusion for training may cause errors from early incorrect predictions to accumulate, damaging the generalization ability of the diagnostic model. MgCF uses the two-tier granularity labels during training. For samples where the pseudo-label from cluster fusion matches the original observed label, fine-grained original labels are used. For samples where they do not match, coarse-grained labels containing information of both classes (i.e. prediction class and observation class) are used, which aims to retain the observed label information to suppress cumulative errors during the training process.

B. Building Multi-granularity clusters

To facilitate the computation of similarity between samples within a cluster and its centroid, and to ensure more significant

differentiation among highly similar samples, we employ an exponential form of cosine similarity to construct the feature similarity function (Eq. (3)).

$$Sim(z^{(i)}, z^{(j)}) = e^{\cos(\theta_{<z^{(i)}, z^{(j)}>})} = e^{\frac{z^{(i)} \times z^{(j)\top}}{|z^{(i)}| \times |z^{(j)}|}} \quad (3)$$

Specifically, for a given feature cluster, MgCF first maps all features in the cluster onto a unit hypersphere. The similarity between features is then represented by the angular distance on this hypersphere, with an exponential function enhancing the differentiation ability, which ensures that the similarity between the sample and its members of the same class is significantly higher than that of samples from other classes, even if the cluster center sample is located at the edge of the distribution of that category.

Additionally, we define a notation rule: let $z_g^{(c)}$ represent the g-th similar feature to feature $z^{(c)}$ (i.e., $\forall i, j \in \mathbb{Z}, i < j \implies Sim(z_i^{(c)}, z^{(c)}) > Sim(z_j^{(c)}, z^{(c)})$). The specific granularity cluster SgC centered at $z^{(c)}$ is defined as:

$$SgC^{(c)} = \{z_i \mid Sim(z_i, z^{(c)}) \geq Sim(z_g^{(c)}, z^{(c)})\} \quad (4)$$

Where $g = \max(G \times \tilde{\eta}, 10)$ represents the cluster's granularity size, determined by the estimated noise intensity $\tilde{\eta}$ of the dataset (initially set to 0, assuming no noisy labels, with the specific estimation method introduced later) and the initial granularity size G (set to 100 in this work).

During training, the value of g for each epoch is dynamically adjusted based on the noise intensity $\tilde{\eta}$ estimated from the previous epoch. Therefore, while the granularity of cluster is specific for each epoch, the overall training process incorporates multi-granularity.

C. Category membership function via cluster fusion

To enhance the robustness and accuracy of determining the class membership of cluster center samples, this paper leverages the evidence combination concept from Dezert-Smarandache Theory [36]. We design a similarity-based cluster fusion process (Eq. (5)), which combines the similarity and label information of samples within a cluster to infer the class membership of the cluster center sample based upon Assumption 1.

$$\begin{cases} \mu(x^{(c)}) & = [\mu_1(x^{(c)}), \cdots, \mu_k(x^{(c)}), \cdots, \mu_K(x^{(c)})] \\ \mu_k(x^{(c)}) & = \frac{\sum_{z^{(i)} \in SgC^{(c)}} Sim(z^{(i)}, z^{(c)}) \cdot \mathbb{1}(\hat{y}^{(c)}=k) \cdot \text{one-hot}(\hat{y}^{(c)})}{\sum_{z^{(i)} \in SgC^{(c)}} Sim(z^{(i)}, z^{(c)})} \end{cases} \quad (5)$$

Specifically, after constructing the feature cluster $SgC^{(c)}$ with a particular granularity, MgCF calculates the membership degree $\mu_k(x^{(c)})$ of the cluster center sample $x^{(c)}$ for each class by integrating information from other samples in the cluster. Using similarity as the belief mass (analogous to weights in weighted fusion), we combine the label information of all samples $z^{(i)}$ within the cluster to determine the support for class k of the cluster center sample. Then, through normalization, MgCF converts these support values into membership degrees, ensuring all values of $\mu(x^{(c)})$ are non-negative and sum to 1, representing the cluster center sample's affiliation to each class.

By employing this similarity-based fusion method, which utilizes information from samples within the cluster, MgCF can improve the robustness and accuracy of class determination for the cluster center sample. Even in the presence of noise and uncertainty, this approach provides a reliable class membership, making the final inferred class membership degree of the cluster center sample $x^{(c)}$ depend not only on its own features but also on the features and similarities of surrounding samples, which leads to more accurate and robust class determinations.

D. Multi-granularity labels and noise intensity estimation

After obtaining the membership function for a sample $x^{(c)}$ through cluster fusion, MgCF needs to defuzzify this membership function $\mu(x^{(c)})$ to generate the corresponding pseudo-label $\bar{y}^{(c)}$ and the confidence of the observed label $\hat{y}^{(c)}$. The pseudo-label is generated using the maximum a posteriori decision rule commonly used in deep learning multi-class tasks, converting the membership function $\mu(x^{(c)})$ into a pseudo-label (Eq. (6)). And the membership degree of the pseudo-label's class is termed the purity of the feature cluster, $Purity^{(c)} = \mu_{\bar{y}^{(c)}}(x^{(c)})$, representing the consistency of labels within the cluster.

$$\bar{y}^{(c)} = \arg\max(\mu(x^{(c)})) \quad (6)$$

Since the purity of a feature cluster reflects the proportion of the majority label within the cluster, we infer that as the model improves its class discrimination ability during training, its latent feature space will increasingly conform to Assumption 1. Therefore, by aggregating the purity information of all feature clusters globally, we can approximate the noise intensity of the current dataset:

$$\tilde{\eta} = 1 - \frac{1}{N} \sum_{i=1}^{N} Purity^{(i)} \quad (7)$$

To determine the confidence $\nu^{(c)}$ of the observed label $\hat{y}^{(c)}$, similar to the belief mass in Dezert-Smarandache Theory, simply using the membership degree of the observed class as the confidence is inadequate. However, this approach neglects the membership degree of the pseudo-label derived from cluster fusion, which is crucial for assessing whether the observed label is a clean label. When the ratio between these degrees is 1, it suggests that the pseudo-label and observed label have similar confidence levels, implying that the observed label is likely clean. Conversely, discrepancies indicate inconsistency among the labels within the cluster, necessitating adjustment of the observed class's membership degree. Thus, MgCF defines the confidence $\nu^{(c)}$ of the observed label $\hat{y}^{(c)}$ as Eq. (8).

$$\nu^{(c)} = \underbrace{\mu_{\hat{y}^{(c)}}(x^{(c)})}_{\text{Membership}} \times \underbrace{\frac{\mu_{\hat{y}^{(c)}}(x^{(c)})}{Purity^{(c)}}}_{\text{Correction}} = \frac{\mu_{\hat{y}^{(c)}}(x^{(c)})^2}{\max(\mu(x^{(c)}))} \quad (8)$$

We then use this adjusted confidence $\nu^{(c)}$ as the weight for the observed class in the multi-granularity label during fusion, ensuring that the fused multi-granularity label sums to one in a one-hot form. MgCF assigns $(1-\nu^{(c)})$, i.e. uncertainty of observed label, as the weight for the pseudo-label.

$$\widetilde{y^{(c)}} = \nu^{(c)} \times \text{One-hot}(\hat{y}^{(c)}) + (1-\nu^{(c)}) \times \text{One-hot}(\bar{y}^{(c)}) \quad (9)$$

According to Definition 1 and 2, assuming Assumption 1 approximately holds, it is evident that the multi-granularity labels generated by MgCF exhibit conditional robustness to noisy labels (as stated in Theorem 1).

Theorem 1: **(Noisy tolerance of MgCF)***For a K-classification task, when there exists a globally optimal model f_g and the model f is trained using cross entropy as the loss function $\ell(p,y) = -\sum_{i=1}^{K} y_i \log(p_i)$, if the noise intensity η in the dataset satisfies $\eta < 1 - \frac{1}{K}$ when the noise type is symmetric (uniform), or satisfies $0 < \eta_{kl} < 1-\eta$ with $\sum_{k\neq l}\eta_{kl} = \eta$ when the noise type is asymmetric (class-dependency), then the fused labels $\widetilde{\mathcal{Y}}$ have noise tolerance.*

Note: In addition to the experimental results and analysis provided later (Sec. IV), a mathematical proof of Theorem 1 is available in the *Sec. s2* of *Supplementary materials*.

E. MgCF for fault diagnosis

Following the current mainstream paradigm of intelligent fault diagnosis, as illustrated in Fig. 2, MgCF's application to fault diagnosis involves three stages: data acquisition, model training, and deployment.

Algorithm 1 Training process based on MgCF

Input: Total number of iterations ($epoch$), the training dataset with noisy labels ($\mathcal{D} = \{\mathcal{X}, \hat{\mathcal{Y}}\}$), estimating noise intensity ($\tilde{\eta}$), and the diagnostic model ($f(\Theta_b, \Theta_c; \cdot)$).
Output: $f(\Theta_b, \Theta_c; \cdot)$ (Trained diagnostic model)
Random initialization backbone (Θ_b) as well as classifier (Θ_c), initialization estimating noise intensity ($\tilde{\eta} = 0$), and initialization train data $\widetilde{\mathcal{D}} = \mathcal{D}$
for e in epoch **do**
  for (x, y) in $\widetilde{\mathcal{D}}$ **do**
    $p \leftarrow f(\Theta_b, \Theta_c; x)$
    $loss \leftarrow \mathcal{L}_{CE}(p, y)$
    Update Θ_b, Θ_c via $loss$
  end for
  Get latent features $\mathcal{Z} = \{z^{(i)} | z^{(i)} = f(\Theta_b; x^{(i)}), \forall x^{(i)} \in \mathcal{X}\}$
  Get feature clusters $SgCs = \{SgC^{(c)}\}_{c=1}^{N} \leftarrow \tilde{\eta}, \mathcal{Z}$ via *Eq.* (4)
  Update estimating noise intensity $\tilde{\eta} \leftarrow SgCs$ via *Eq.* (7)
  Get Multi-granularity labels $\widetilde{\mathcal{Y}} \leftarrow SgCs$ via *Eq.* (9) i.e. cluster fusion
  Update train data $\widetilde{\mathcal{D}} = \{\mathcal{X}, \widetilde{\mathcal{Y}}\}$
end for

The stage of data acquisition involves collecting monitoring signals from operational mechanical systems. These signals are then annotated, either fully automatically or semi-automatically, to create a dataset for training. The training process is a standard supervised learning procedure that uses the cross-entropy loss function, a common choice for multi-class classification tasks, to calculate loss values, perform gradient computation, and update model parameters. Due to the low-cost annotation, it is difficult to avoid noisy labels. Therefore, multi-granularity labels are constructed based on the predictions of model at each epoch during training (as detailed in Alg. 1 and Alg. s1 of *Sec. s3*) to mitigate the impact of noisy labels on the generalization. Finally, the trained diagnostic model is deployed in industrial settings, e.g. edge computing devices [37], to perform real-time fault diagnosis on operational machinery.

IV. Experimental verification and discussion

A. Experimental settings

1) Dataset description: With the aim of validating the superiority of MgCF and analyzing the sources of its effectiveness, we conducted a series of experiments on a self-made bearing damage diagnosis dataset (see Table I), followed by a detailed analysis and discussion of the results. To further enhance the credibility of our findings, we replicated the experiments with identical settings on an additional *open-source* multi-bearing fault diagnosis dataset [5], and the results are provided in the *supplementary materials* as secondary evidence.

TABLE I
Detailed information of the Damage datasets

	Information
Frequency	16 kHz
Motor speed	[1770, 1775], [2366, 2370], and [2959, 2962] rpm
Load	0.0 NM and 0.3 NM
Fault diamete	0.2 mm, 0.4 mm, 0.6 mm, and 0.8 mm
Fault location	I, O, B, I&O, O&B, and B&I
No. of categories	$1 + 6 \times 4 = 25$ ("1" refers to normal bearings)
No. of samples	$25 \times 6 \times 640 = 96,000$

Note: Where, "I" indicates that the fault is located on the inner ring, "O" represents that fault occurred the outer ring, and "B" means that the ball of bearing is faulty. In this work, faults that occur at the same location but have varying degrees of damage are considered different categories, and the signals of similar faults under different working conditions are considered to be of a category.

The two datasets used in this work consist of vibration signals collected from mechanical systems during operation, representing the operational states of the bearings under diagnosis. Each sample is a one-dimensional time series of 2048 consecutive sampling points randomly extracted from the raw vibration signals. The datasets are categorized into six different operating conditions based on load and speed, with an equal number of samples for each fault type (i.e., this work does not consider imbalanced conditions). All datasets were uniformly divided into training and testing sets in a 1:1 ratio.

Additionally, the labels in the training set were modified to introduce varying levels of symmetric and asymmetric noise, creating training sets with different noise intensities. Here, with the motivation to simulate non class related symmetric noise that may occur during the automatic annotation process, asymmetric noise is introduced by swapping labels of similar classes based on a specific noise ratio η_{ij}. For example, swapping "0.2 mm fault on outer (O 0.2)" and "0.4 mm fault on outer (O 0.4)" instead of "fault on outer" and damage at other locations. For more detailed swapping information, please refer to the *Table s2 and s3*.

2) Optimization process: In this work, we used the AdamW optimizer integrated in *PyTorch* for training the diagnostic model (*Table s4*). Before formal training, the model underwent a five-epoch warm-up phase, followed by 100 epochs of formal training. The initial learning rate was set to $3 \times 10^{-3} \times bs/512$ (where bs denotes the batch size during training) and was gradually reduced to 0 using cosine annealing.

To avoid misleading results caused by random initialization, all reported experimental results are the statistical results of at least ten repeated experiments, with all detailed accuracy distributions available in the *Supplementary materials*.

Besides, to validate the effectiveness of MgCF, we employed the recently released diagnostic framework MSACNN [38] as the backbone. Meanwhile, to avoid introducing specific errors due to different model architectures, we provided supplementary results using another model, MgNet [5] who is a diagnostic framework released alongside the open-source dataset used in this work, as the backbone.

Note: In subsequent experiments, only the results of MSACNN on the *Damage* dataset were displayed, while the experimental results of MSACNN on the *Multiple* dataset and MgNet on the *Damage* and *Multiple* datasets were summarized in *Supplementary materials*.

B. Case I: Effectiveness of MgCF

To validate the effectiveness and superiority of MgCF, in ten different noisy environments, we tested its performance along with eight advanced learning with noisy labels methods, including SCE (2019 [18]), JoCoR (2020 [27]), RRL (2021 [39]), UNICON (2022 [20]), AGCE (2023 [21]), SPRL (2023 [40]), RML (2024 [12]), and LSL (2024 [11]), and the experimental results are summarized in Fig. 4, with detailed results provided in *Tables s7*, and *s8*.

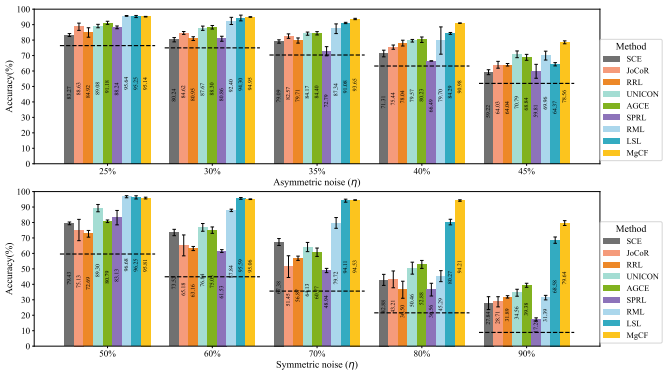


Fig. 4. Comparison of the effectiveness of MgCF and eight other methods under varying intensities of asymmetric and symmetric noise. The dashed lines in the figure represent the baseline results, which are the average accuracies obtained by training directly with the dataset's observed labels using cross-entropy as the loss function, without any additional noise-robust techniques.

Among these methods, SCE, AGCE, SPRL, and RML utilize strategies that correct loss functions/values to mitigate the interference of noisy labels. JoCoR is a representative method for co-training-based noisy label learning. UNICON builds on co-training by introducing a unified selection mechanism to address the imbalance between easy and hard samples. RRL combats noise by learning robust representations based on the geometric structure of the regularized feature subspace, while LSL further embeds the structural information of the feature space into the labels to enhance generalization.

The experimental results clearly demonstrate that MgCF achieves highly competitive performance under various noise conditions. In low-noise environments (i.e., asymmetric noise intensity below 40% and symmetric noise intensity below 80%), MgCF achieved the highest or second-highest accuracy, with the difference between the second-best and the best methods being within 1%, indicating strong competitiveness.

However, in high-noise environments (i.e., asymmetric noise intensity above 40% and symmetric noise intensity above 80%), MgCF significantly outperformed other advanced noisy label learning methods, with some cases showing more than a ten-percentage-point improvement in accuracy. For example, under 90% symmetric noise, MgCF achieved an accuracy of 79.64±1.56%, while the next best method, LSL, achieved only 68.58±2.05%. These results highlight MgCF's ability to effectively handle high levels of noise.

MgCF's consistently high performance across different noise rates and types (asymmetric and symmetric) demonstrates its robustness and generalization ability, which makes it a promising approach for real-world applications where noisy labels are prevalent. Additionally, the detailed comparisons in *Tables s7* and *s8* further validate MgCF's superior performance, showing that its effectiveness and superiority are not limited to specific backbone networks and diagnostic tasks, and highlighting its potential as a state-of-the-art method for fault diagnosis with noisy labels.

C. Case II: Ablation for MgCF

With the motivation to further analyze the sources of MgCF's effectiveness, we constructed and evaluated four different variants of MgCF to discuss the role of correcting the confidence of observed labels during defuzzification to generate pseudo labels and the necessity of using multi-granularity labels for training etc.

The experimental results (Fig. 5), detailed in *Table s9*, provide insights into the role of correcting the confidence of observed labels and the necessity of using multi-granularity labels for training, i.e., the ablation experiment confirms that each component of MgCF plays a vital role in its overall performance. Correcting the confidence of observed labels, utilizing multi-granularity labels, and the fusion process collectively enhance the robustness and accuracy of the model.

1) Impact of "Correction" term in Eq.(8) : Removing the correction term (M1) resulted in a notable decline in accuracy across all noise levels for both asymmetric and symmetric noise. Specifically, under 45% asymmetric noise, accuracy dropped to 71.71% compared to 78.56% with the full MgCF model. Similarly, under 90% symmetric noise, the accuracy decreased to 64.26%. This demonstrates that correcting the confidence of observed labels significantly contributes to the robustness and accuracy of the model. Replacing the confidence of observed labels with category membership $\mu_{\widehat{\mathcal{Y}}}(\widetilde{X})$ (M2) also led to reduced performance, particularly under higher noise levels. At 45% asymmetric noise, accuracy fell to 54.63%, and under 90% symmetric noise, it dropped

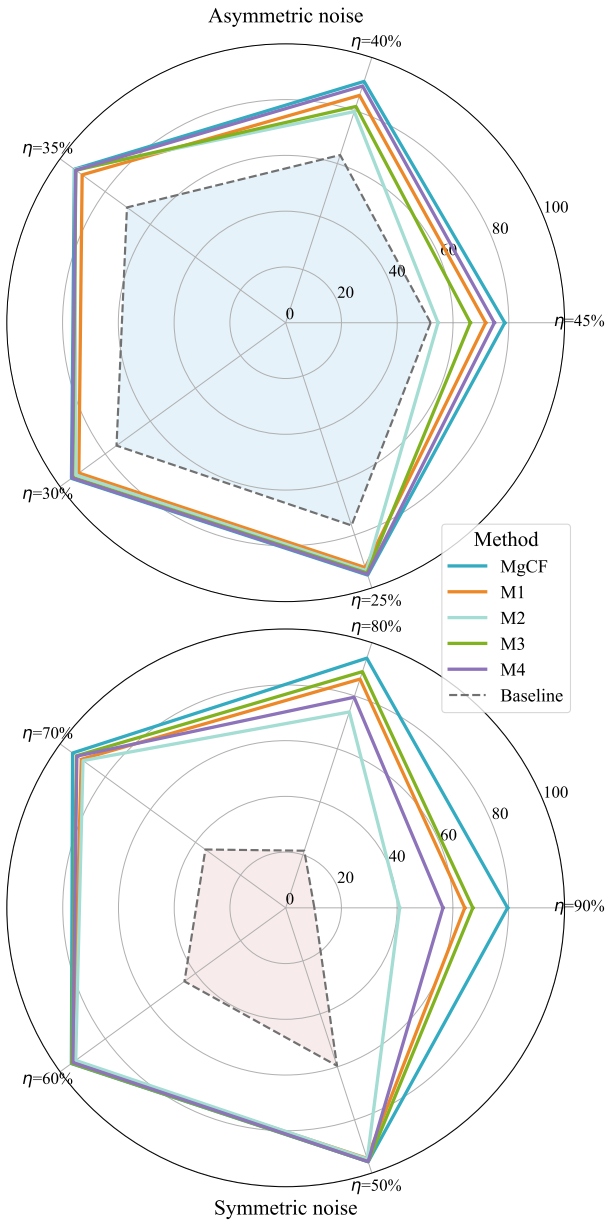


Fig. 5. The experimental results after ablation of different modules, where M1-4 refers to the four variant structures of MgCF. Specifically, M1 refers to removing the "Correction" term in Eq. (8). M2 involves replacing the confidence of the observed labels $\widetilde{V}$ with the category membership $\mu_{\widetilde{\mathcal{Y}}}(\widetilde{X})$ in the fusion process of Eq. (9). M3 means directly using the pseudo-labels $\overline{\mathcal{Y}}$ instead of the fused labels $\widetilde{\mathcal{Y}}$ for training. M4 replaces the fused labels with the category membership $\mu(\widetilde{X})$ to perform the learning process.

to 40.89%, which indicates that while category membership provides some level of robustness, the confidence correction mechanism is crucial for maintaining higher accuracy, especially in highly noisy environments.

2) Effect of directly using pseudo labels: Directly using pseudo-labels instead of fused labels (M3) demonstrates the effectiveness of cluster-based pseudo-labels in combating noisy labels. However, M3 is prone to cumulative errors in high-noise environments. For instance, with 45% asymmetric noise, the accuracy is 66.22%, and with 90% symmetric noise, the accuracy is 67.16%. These results indicate that while cluster fusion based pseudo labels are effective against noisy labels, relying solely on single-granularity pseudo-labels means discarding the information from observed labels. Consequently, this approach entirely depends on the model's predictions during training, which can lead to significant accuracy degradation in high-noise environments. In contrast, it maintains the robustness of fused labels in low-noise settings.

3) Learning with category membership: Replacing fused labels with class membership (M4) shows better performance than M1 and M2 but still falls short of the complete MgCF model. With 45% asymmetric noise, the accuracy is 74.83%, and with 90% symmetric noise, the accuracy is 56.45%. The performance decline in M4 is likely due to using membership values directly as multi-granularity pseudo-labels. Although this approach uses clustering information for label correction, it overly focuses on multiple classes during training. A critical requirement is to assess whether the original observed label is clean or noisy and whether the cluster-based pseudo-label is reliable. Consequently, M4 is more susceptible to misleading noisy labels in high-noise environments, highlighting the rationale behind the multi-granularity label construction method in Eq. (9).

D. Limitations analysis

1) High-noise environments: Assumption 1 Breakdown: MgCF demonstrates significant robustness under moderate noise levels, as evidenced by the probability mapping (Fig. 3) and t-SNE visualization (Fig. s5). However, in extreme noise conditions—such as when symmetric noise intensity reaches 90%—the validity of Assumption 1 is compromised. Specifically, as feature clusters increasingly overlap, the correlation between feature similarity and label consistency diminishes. This degradation weakens MgCF's ability to leverage the inherent structure of the latent space.

Despite this limitation, MgCF's multi-granularity label mechanism mitigates the effects of extreme noise by introducing coarse-grained labels for low-confidence pseudo-labels. This enables the model to incorporate uncertainty into the label generation process and avoids prematurely relying on potentially incorrect pseudo-labels, as discussed in Sec. s2.1 of the *Supplementary materials*. Nonetheless, improving the label construction process for such challenging scenarios remains a topic for future exploration.

2) Overfitting in low-noise environments: In low-noise environments, MgCF exhibits a tendency to overfit during the later stages of training. This can be attributed to an over-reliance on the cluster center-based category membership function, which reduces confidence in noisy labels and undermines the advantages of multi-granularity labels. While certain data augmentation strategies, such as jitter and noise addition, alleviate this issue and even enhance MgCF's performance under high-noise conditions, their effectiveness in low-noise scenarios is limited. The lack of augmentation techniques tailored specifically for MgCF underscores the need for developing methods aligned with its underlying principles.

Moreover, MgCF inherently requires a sufficiently large training dataset to perform effectively. As highlighted in *Sec. s1.4.1*, reducing the training set size below 60–70% significantly impacts performance. This dependency emphasizes the

importance of future approaches addressing scenarios where training data availability is constrained.

V. Conclusion

With the goal of addressing the question "how to maximize the use of 'noisy' sample label information to avoid cumulative errors during self-guided learning, and to prevent discarding 'noisy' labels during training, which could waste valuable data and potentially useful information," this work proposed a **M**ulti-**g**ranularity **C**lusters **F**usion (*MgCF*) approach, which dynamically constructs feature clusters and integrates label information to generate noise-tolerant multi-granularity labels, thus achieving more robust training of DNNs and enhancing their generalization without needing to know the exact proportion of noisy labels. We validated the effectiveness and superiority of MgCF in fault diagnosis tasks, and a series of experimental results demonstrate that MgCF has the ability to create structured and coherent feature space clusters, significantly improving the practical diagnostic accuracy, and MgCF promises to offer a more robust and promising training paradigm for a range of potential applications, including fault diagnosis.

While MgCF provides a robust framework for fault diagnosis with noisy labels, its performance under extreme noise conditions and its tendency to overfit in low-noise settings highlight areas for improvement. Future research could focus on enhancing cluster-based label generation and developing augmentation strategies tailored specifically to MgCF. Moreover, creating methods to improve performance in data-constrained scenarios would expand MgCF's applicability to a wider range of real-world tasks.

References

[1] H. Fang, J. Xiang, J. An, H. Liu, H. Li, Y. Cui, and F. Dunkin, "Fcdg: A central dogma-inspired approach for cross-domain fault diagnosis," *IEEE Sensors Journal*, vol. 25, no. 3, pp. 5192–5199, 2025.

[2] R. Huang, J. Li, Y. Liao, J. Chen, Z. Wang, and W. Li, "Deep adversarial capsule network for compound fault diagnosis of machinery toward multidomain generalization task," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–11, 2021.

[3] J. Chen, K. Wen, J. Xia, R. Huang, Z. Chen, and W. Li, "Knowledge embedded autoencoder network for harmonic drive fault diagnosis under few-shot industrial scenarios," *IEEE Internet of Things Journal*, vol. 11, no. 13, pp. 22 915–22 925, 2024.

[4] S. Yang, B. Lei, Q. Wang, J. Chang, X. Kong, and H. Cheng, "Dual weighted-class adversarial network for rotary machine fault diagnosis using multisource domain with class-inconsistent data," *IEEE/ASME Transactions on Mechatronics*, pp. 1–12, 2024.

[5] J. Deng, H. Liu, H. Fang, S. Shao, D. Wang, Y. Hou, D. Chen, and M. Tang, "MgNet: A fault diagnosis approach for multi-bearing system based on auxiliary bearing and multi-granularity information fusion," *Mechanical Systems and Signal Processing*, vol. 193, p. 110253, 2023.

[6] H. Fang, J. An, H. Liu, J. Xiang, B. Zhao, and F. Dunkin, "A lightweight transformer with strong robustness application in portable bearing fault diagnosis," *IEEE Sensors Journal*, vol. 23, no. 9, pp. 9649–9657, 2023.

[7] F. Dunkin, X. Li, C. Hu, G. Wu, H. Li, X. Lu, and Z. Zhang, "Like draws to like: A multi-granularity ball-intra fusion approach for fault diagnosis models to resists misleading by noisy labels," *Advanced Engineering Informatics*, vol. 60, p. 102425, 2024.

[8] J. Yuan, R. Zhao, K. Wei, P. Chen, and T. He, "Fault diagnosis of multichannel bearing-rotor system via multistructure collaborative discriminative embedding," *IEEE/ASME Transactions on Mechatronics*, pp. 1–12, 2024.

[9] W. Li, H. Lan, J. Chen, K. Feng, and R. Huang, "Wavcapsnet: An interpretable intelligent compound fault diagnosis method by backward tracking," *IEEE Transactions on Instrumentation and Measurement*, vol. 72, pp. 1–11, 2023.

[10] W. Li, R. Huang, J. Li, Y. Liao, Z. Chen, G. He, R. Yan, and K. Gryllias, "A perspective survey on deep transfer learning for fault diagnosis in industrial scenarios: Theories, applications and challenges," *Mechanical Systems and Signal Processing*, vol. 167, p. 108487, 2022.

[11] N.-r. Kim, J.-S. Lee, and J.-H. Lee, "Learning with structural labels for learning with noisy labels," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2024, pp. 27 610–27 620.

[12] F. Li, K. Li, J. Tian, and J. Zhou, "Regroup median loss for combating label noise," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 12, 2024, pp. 13 474–13 482.

[13] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov *et al.*, "The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale," *International journal of computer vision*, vol. 128, no. 7, pp. 1956–1981, 2020.

[14] D. Mahajan, R. Girshick, V. Ramanathan, K. He, M. Paluri, Y. Li, A. Bharambe, and L. Van Der Maaten, "Exploring the limits of weakly supervised pretraining," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 181–196.

[15] F. Dunkin, X. Li, H. Li, G. Wu, C. Hu, and S. S. Ge, "Mgcnl: A sample separation approach via multi-granularity balls for fault diagnosis with the interference of noisy labels," *IEEE Transactions on Automation Science and Engineering*, vol. Early Access, pp. 1–14, 2024.

[16] C. Cheng, X. Liu, B. Zhou, and Y. Yuan, "Intelligent fault diagnosis with noisy labels via semisupervised learning on industrial time series," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 6, pp. 7724–7732, 2023.

[17] Z. Zhu, J. Wang, and Y. Liu, "Beyond images: Label noise transition matrix estimation for tasks with lower-quality features," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 27 633–27 653.

[18] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, and J. Bailey, "Symmetric cross entropy for robust learning with noisy labels," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 322–330.

[19] S. Li, X. Xia, H. Zhang, Y. Zhan, S. Ge, and T. Liu, "Estimating noise transition matrix with label correlations for noisy multi-label learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 184–24 198, 2022.

[20] N. Karim, M. N. Rizve, N. Rahnavard, A. Mian, and M. Shah, "Unicon: Combating label noise through uniform selection and contrastive learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 9676–9686.

[21] X. Zhou, X. Liu, D. Zhai, J. Jiang, and X. Ji, "Asymmetric loss functions for noise-tolerant learning: Theory and applications," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 7, pp. 8094–8109, 2023.

[22] S. Li, X. Xia, S. Ge, and T. Liu, "Selective-supervised contrastive learning with noisy labels," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 316–325.

[23] L. Jiang, Z. Zhou, T. Leung, L.-J. Li, and L. Fei-Fei, "MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 10–15 Jul 2018, pp. 2304–2313.

[24] X. Xia, T. Liu, N. Wang, B. Han, C. Gong, G. Niu, and M. Sugiyama, "Are anchor points really indispensable in label-noise learning?" *Advances in neural information processing systems*, vol. 32, 2019.

[25] T. Liu and D. Tao, "Classification with noisy labels by importance reweighting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 3, pp. 447–461, 2016.

[26] X. Xia, T. Liu, B. Han, N. Wang, M. Gong, H. Liu, G. Niu, D. Tao, and M. Sugiyama, "Part-dependent label noise: Towards instance-dependent label noise," *Advances in Neural Information Processing Systems*, vol. 33, pp. 7597–7610, 2020.

[27] H. Wei, L. Feng, X. Chen, and B. An, "Combating noisy labels by agreement: A joint training method with co-regularization," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 13 726–13 735.

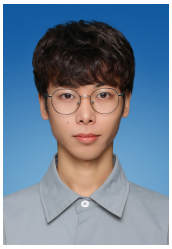
[28] J. Zhong, Y. Yang, H. Mao, A. Qin, X. Li, and W. Tang, "Contrastive regularization guided label refurbishment for fault diagnosis under label noise," *Advanced Engineering Informatics*, vol. 61, p. 102478, 2024.
[29] J. Wei, H. Liu, T. Liu, G. Niu, M. Sugiyama, and Y. Liu, "To smooth or not? when label smoothing meets noisy labels," in *International Conference on Machine Learning*. PMLR, 2022, pp. 23 589–23 614.
[30] A. Vahdat, "Toward robustness against label noise in training deep discriminative neural networks," *Advances in neural information processing systems*, vol. 30, 2017.
[31] K.-H. Lee, X. He, L. Zhang, and L. Yang, "CleanNet: Transfer learning for scalable image classifier training with label noise," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 5447–5456.
[32] Y. Shen and S. Sanghavi, "Learning with bad training data via iterative trimmed loss minimization," in *International conference on machine learning*. PMLR, 2019, pp. 5739–5748.
[33] P. Wu, S. Zheng, M. Goswami, D. Metaxas, and C. Chen, "A topological filter for learning with label noise," *Advances in neural information processing systems*, vol. 33, pp. 21 382–21 393, 2020.
[34] C. Feng, G. Tzimiropoulos, and I. Patras, "Ssr: An efficient and robust framework for learning with unknown label noise," *arXiv preprint arXiv:2111.11288*, 2021.
[35] Y. Yao, Z. Sun, C. Zhang, F. Shen, Q. Wu, J. Zhang, and Z. Tang, "Jo-src: A contrastive approach for combating noisy labels," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 5192–5201.
[36] L. Xinde, F. Dunkin, and J. Dezert, "Multi-source information fusion: Progress and future," *Chinese Journal of Aeronautics*, vol. 37, no. 7, pp. 24–58, 2024.
[37] H. Fang, J. An, B. Sun, D. Chen, J. Bai, H. Liu, J. Xiang, W. Bai, D. Wang, S. Fan *et al.*, "Empowering intelligent manufacturing with edge computing: A portable diagnosis and distance localization approach for bearing faults," *Advanced Engineering Informatics*, vol. 59, p. 102246, 2024.
[38] X. Yuan, L. Huang, L. Ye, Y. Wang, K. Wang, C. Yang, W. Gui, and F. Shen, "Quality prediction modeling for industrial processes using multiscale attention-based convolutional neural network," *IEEE Transactions on Cybernetics*, vol. 54, no. 5, pp. 2696–2707, 2024.
[39] J. Li, C. Xiong, and S. C. Hoi, "Learning from noisy data with robust representation learning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 9485–9494.
[40] X. Shi, Z. Guo, K. Li, Y. Liang, and X. Zhu, "Self-paced resistance learning against overfitting on noisy labels," *Pattern Recognition*, vol. 134, p. 109080, 2023.

Fir Dunkin(Graduate Student Member IEEE) received the M.E. degree from the School of Automation Engineering, Northeast Electric Power University, Jilin, China, in 2022.

Currently, he is pursuing the Ph.D. degree with the Southeast University, and his research interests include Information fusion, Mechatronics, and Uncertainty reasoning etc. Meanwhile, he has published over 20 peer-reviewed papers, including multiple *ESI* highly cited papers, and submitted over 100 peer-reviewed results as a reviewer for journals including IEEE TFS/TII/TCSVT/TMECH/TIM, InfFus, MSSP, PR, NeuNet, KBS, ESWA, EAAI, etc.

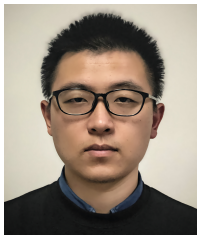
Xinde Li(Senior Member IEEE) received his Ph.D. degree in control theory and control engineering from the Department of Control Science and Engineering, Huazhong University of Science and Technology, Wuhan, China, in 2007. After that, he joined the School of Automation, Southeast University, Nanjing, China, where he is currently a professor. He was elected as a senior member of Institute of Electrical and Electronics Engineers in 2016, an academician with the Russian Academy of Natural Sciences and a cover person of Scientific Chinese in 2021 and a fellow of the Institution of Engineering and Technology in 2022. He is the vice director of Intelligent Robot Committee of the Chinese Association for Artificial Intelligence from 2017, the vice director of Intelligent Products and Industry Working Committee of the Chinese Association for Artificial Intelligence from 2019 and the vice director of Intelligent Manufacturing Systems & Technology Professional Committee of Chinese Association of Automation from 2022.

His research interests include intelligent robot, and unmanned system, human-machine interaction, computer vision, information fusion. He was the principal of about 50 projects including key and major projects of NSFC, etc. He published more than 100 papers, 5 books and won 32 national invention patents. He also won many prizes including global award for outstanding scientist in artificial intelligence and robotics, Asia award for outstanding researcher, gold award in Geneva international invention, etc. He was also the associate editors of several reputative international journals, i.e. IEEE Transactions on Fuzzy Systems, Journal of Systems Engineering and Electronics, etc.



Guoliang Wu received the B.E. degree from the School of Information Science and Technology, Beijing University of Chemical Technology, Beijing, China, in 2020.

At present, he is pursuing the Ph.D. degree with the School of Automation, Southeast University, Nanjing, China. His current research interests include Multi-granular information fusion, Intelligent Game, Neural networks, and Pattern recognition etc.



Chuanfei Hu(Graduate Student Member IEEE) received the M.S. degree from the School of Optical-Electrical and Computer Engineering, University of Shanghai for Science and Technology, Shanghai, China, in 2021.

He is currently pursuing the Ph.D. degree in electronic information engineering with the School of Automation, Southeast University, Nanjing, China. His research interests include deep learning, industrial inspection, attribute learning, and affective computing.

Le Yu received a Bachelor of Engineering in Automation from the School of Information Science and Engineering at Northeastern University, Shenyang, China, in 2022. He is currently pursuing a Ph.D. in Automation at the Department of Automation, Southeast University, Nanjing, China. His research interests include machine learning, image processing, knowledge representation, and reasoning.

Xiaoyan Lu received the B.S. and M.S. degrees in measurement and control technology from the Anhui University of Technology, Ma'anshan, China, in 2015 and 2018, respectively. She is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Southeast University, Nanjing, China. Her research interests include computer vision, machine learning, and surveillance video analysis.

Shuzhi Sam Ge (Fellow IEEE) Fellows of SAEng, IEEE, IFAC, IET, CAA, Professor, Department of Electrical and Computer Engineering, PI Member of Institute for Functional Intelligent Materials, the National University of Singapore. Founder of Institute for Future (IFF), Qingdao University, China. He Serves as President Elect of Asian Control Association, 2022-2024, IFAC executive officers as Council Member, 2023-2026, Steering Committee Chair of International Conference on Social Robotics. He served as Vice President of Technical Activities and Membership Activities, 2009-2012, Member of Board of Governors, 2007-2009, and Chair of Technical Committee on Intelligent Control, 2005-2008, of IEEE Control Systems Society. He was the recipient of many award including National Technology Award from Singapore, IEEE Control Systems Society Distinguished Member Award, AI Grand Challenge Award from AI SG. His research interests include robotics, intelligent systems, and intelligent materials. He has (co)-authored nine books, and over 800 international journal and conference papers, with high H index (110) and citations (58,000).

Supporting materials

Wisdom via Multiple Perspectives: A Multi-granularity Clusters Fusion Approach for Fault Diagnosis with Noisy Labels

Fir Dunkin, Xinde Li, Guoliang Wu, Chuanfei Hu, Le Yu, Xiaoyan Lu, and Shuzhi Sam Ge

S1 SUPPLEMENTARY INFORMATION

s1.1 Implementation details

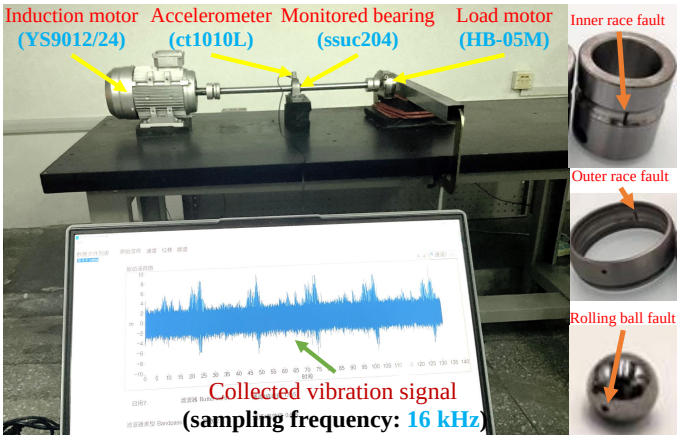


Fig. s1: The experimental rig used in constructing the self-made bearings fault diagnosis dataset, i.e. *Damage* dataset.

The *Multiple* dataset, used for additional validation, is a multi-bearing fault diagnosis dataset that includes data for both proximal and distal bearings under various conditions. For this study, the Multiple dataset includes four types of conditions for both the proximal and distal bearings: normal and faults in the inner race, outer race, and ball. All faults are characterized by gaps with a depth of 0.2 mm and a width and length of 0.1 mm. Thus, *Multiple* is a 16-class fault diagnosis dataset.

- *Fir Dunkin, Xinde Li, Guoliang Wu, Chuanfei Hu, Le Yu, and Xiaoyan Lu are with the School of Automation, Southeast University, China (Emails: dunkin2fir@ieee.org; xindeli@seu.edu.cn; 230208677@seu.edu.cn; chuanfei_hu@ieee.org; leyu@seu.edu.cn; 15850500060@163.com)*
  Fir Dunkin, Xinde Li, Chuanfei Hu, and Le Yu also are with the Key Laboratory of Measurement and Control of CSE, Ministry of Education, China.
  Xinde Li also is with the Southeast University Shenzhen Research Institute, China.
  Shuzhi Sam Ge is with the Social Robotics Laboratory, Department of Electrical and Computer Engineering, Interactive Digital Media Institute, National University of Singapore, Singapore.

*This work was supported in part by the National Natural Science Foundation of China under Grant 62233003 and 62073072, the Key Projects of Key R&D Program of Jiangsu Province under Grant BE2020006 and BE2020006-1, and the Shenzhen Science and Technology Program under Grant JCYJ20210324132202005 and JCYJ20220818101206014 (*Corresponding author: Xinde Li).*

TABLE s1: Detailed information of the *Multiple* datasets

	Information
Frequency	12 kHz
Motor speed	[1770, 1775], [2366, 2370], and [2959, 2962] rpm
Load	0.0 NM and 0.3 NM
Fault location	Inner race, Outer race, and Ball
No. of categories	$(1+3+3)\times(1+3+3)=49$ ("1" refers to normal bearings)
No. of samples	$16\times6\times680=65,280$

TABLE s2: Swapping categories for *Damage* dataset

Category 1	swapping	Category 2
Inner (0.2 mm)	⇌	Inner (0.4 mm)
Inner (0.6 mm)	⇌	Inner (0.8 mm)
Outer (0.2 mm)	⇌	Outer (0.4 mm)
Outer (0.6 mm)	⇌	Outer (0.8 mm)
Ball (0.2 mm)	⇌	Ball (0.4 mm)
Ball (0.6 mm)	⇌	Ball (0.8 mm)
Inner & Outer (0.2 mm)	⇌	Inner & Outer (0.4 mm)
Inner & Outer (0.6 mm)	⇌	Inner & Outer (0.8 mm)
Outer & Ball (0.2 mm)	⇌	Outer & Ball (0.4 mm)
Outer & Ball (0.6 mm)	⇌	Outer and Ball (0.8 mm)
Ball & Inner (0.2 mm)	⇌	Ball & Inner (0.4 mm)
Ball & Inner (0.6 mm)	⇌	Ball & Inner (0.8 mm)

TABLE s3: Swapping categories for Multiple dataset

Category 1	swapping	Category 2
Proximal normal & Distal inner	⇌	Proximal inner & Distal normal
Proximal normal & Distal outer	⇌	Proximal outer & Distal normal
Proximal normal & Distal ball	⇌	Proximal ball & Distal normal
Proximal inner & Distal normal	⇌	Proximal normal & Distal inner
Proximal inner & Distal outer	⇌	Proximal outer & Distal inner
Proximal inner & Distal ball	⇌	Proximal ball & Distal inner
Proximal outer & Distal normal	⇌	Proximal normal & Distal outer
Proximal outer & Distal inner	⇌	Proximal inner & Distal outer
Proximal outer & Distal ball	⇌	Proximal ball & Distal outer
Proximal ball & Distal normal	⇌	Proximal normal & Distal ball
Proximal ball & Distal inner	⇌	Proximal inner & Distal ball
Proximal ball & Distal outer	⇌	Proximal outer & Distal ball

TABLE s4: The implementation details

Config	Value
Backbone	MgNet and MSACNN
Optimizer	AdamW
Initial learning rate	$3\times10^{-3}\times\frac{batchsize}{512}$
Batch size	512
Weight decay	10^{-2}
Optimizer momentum	$\beta_1=0.9$, $\beta_2=0.999$
Learning rate schedule	CosineAnnealingLR
Warm-up epochs	5
Training epochs	100
GPU	NVIDIA A100
Pytorch version	2.5.1

s1.2 Detailed information of diagnostic model

TABLE s5: Detailed architecture information of MgNet

Layer (Type: Depth)	Input Shape	Output Shape	Param #	Mult-Adds
MgNet: 1-1 (**Backbone**)	[-1, 1, 2048]	[-1, 64]	2	-
Sequential: 2-1	[-1, 1, 2048]	[-1, 128, 8]	-	-
MgFF: 3-1	[-1, 1, 2048]	[-1, 4, 512]	218	131,112
MgFF: 3-2	[-1, 4, 512]	[-1, 16, 128]	2,582	475,296
MgFF: 3-3	[-1, 16, 128]	[-1, 64, 32]	38,678	1,852,032
MgFF: 3-4	[-1, 64, 32]	[-1, 128, 8]	222,742	3,024,128
Linear: 2-2	[-1, 128]	[-1, 64]	-	-
Linear: 3-5	[-1, 128]	[-1, 64]	8,256	8,256
Linear: 1-2 (**Predictor**)	[-1, 64]	[-1, 25]	1,625	1,625
Total params: 274,103 **Trainable params: 274,103** **Non-trainable params: 0** **Total mult-adds (Units.MEGABYTES): 5.49**				
Input size (MB): 0.01 **Forward/backward pass size (MB): 1.43** **Params size (MB): 1.10** **Estimated Total Size (MB): 2.54**				

TABLE s6: Detailed architecture information of MSACNN

Layer (Type: Depth)	Input Shape	Output Shape	Param #	Mult-Adds
MSACNN: 1-1 (**Backbone**)	[-1, 1, 2048]	[-1, 64]	-	-
MultiScaleConv: 2-1	[-1, 1, 32, 64]	[-1, 96, 32, 64]	2,752	-
ChannelAttention: 2-2	[-1, 96, 32, 64]	[-1, 96, 1, 1]	1,152	1,152
MaxPool2d: 2-3	[-1, 96, 32, 64]	[-1, 96, 16, 32]	-	-
MultiScaleConv: 2-4	[-1, 96, 16, 32]	[-1, 192, 16, 32]	510,144	-
ChannelAttention: 2-5	[-1, 192, 16, 32]	[-1, 192, 1, 1]	4,608	4,608
MaxPool2d: 2-6	[-1, 192, 16, 32]	[-1, 192, 8, 16]	-	-
Sequential: 2-7	[-1, 192, 8, 16]	[-1, 12288]	-	-
Linear: 2-8	[-1, 12288]	[-1, 64]	786,496	786,496
Linear: 2-9 (**Predictor**)	[-1, 64]	[-1, 25]	1,625	1,625
Total params: 1,306,777 **Trainable params: 1,306,777** **Non-trainable params: 0** **Total mult-adds (Units.MEGABYTES): 267.63**				
Input size (MB): 0.01 **Forward/backward pass size (MB): 2.36** **Params size (MB): 5.23** **Estimated Total Size (MB): 7.60**				

s1.3 Supplementary experimental results

TABLE s7: Comparison of effectiveness between MgCF and other methods for for learning with noisy labels. (The backbone, i.e. the encoder in Fig.2, is *MSACNN*)

Datasets	Methods	Accuracy(%) under **asymmetric** noise with different rates					Accuracy(%) under **symmetric** noise with different rates				
		$\eta = 25\%$	$\eta = 30\%$	$\eta = 35\%$	$\eta = 40\%$	$\eta = 45\%$	$\eta = 50\%$	$\eta = 60\%$	$\eta = 70\%$	$\eta = 80\%$	$\eta = 90\%$
Damage	Baseline	76.43±1.17	74.97±0.31	70.35±1.84	63.22±1.69	51.98±1.13	59.61±1.23	44.89±0.85	35.62±2.53	21.56±1.45	10.30±1.37
	SCE (2019)	83.27±0.93	80.24±1.39	79.09±1.09	71.31±2.16	59.22±1.60	79.43±0.88	73.52±2.06	67.38±2.24	42.88±3.57	27.84±4.16
	JoCoR (2020)	88.63±2.35	84.62±0.93	82.57±1.44	75.44±1.37	64.03±2.21	75.13±6.89	65.18±6.76	51.45±7.03	43.21±5.43	28.71±3.29
	RRL (2021)	84.92±3.05	80.95±1.22	79.71±1.68	78.04±1.88	64.04±0.82	72.69±2.17	63.16±1.33	56.87±1.38	36.50±5.51	31.89±0.77
	UNICON (2022)	89.08±1.10	87.67±1.38	84.17±1.02	79.57±0.84	70.79±2.18	89.30±2.37	76.84±2.50	64.13±2.98	50.46±3.87	34.56±2.25
	AGCE (2023)	91.18±1.00	88.30±1.14	84.40±1.06	80.23±1.78	68.84±1.92	80.79±0.84	75.05±2.07	60.77±2.70	52.88±2.58	39.38±1.31
	SPRL (2023)	88.24±0.89	80.86±1.62	72.79±2.96	66.49±0.05	59.81±4.51	83.13±4.66	61.53±0.92	48.94±1.37	36.56±4.12	17.28±1.04
	RML (2024)	**95.64±0.26**	92.40±2.32	87.34±3.13	79.70±8.82	69.96±2.87	**96.68±0.68**	87.84±0.78	79.72±3.48	45.29±3.56	31.39±1.47
	LSL (2024)	95.25±0.69	94.30±1.86	91.08±0.42	84.29±0.63	64.37±1.06	96.25±1.04	**95.59±0.58**	94.11±0.97	80.27±1.81	68.58±2.05
	MgCF (**Ours**)	95.14±0.20	**94.95±0.17**	**93.65±0.43**	**90.98±0.05**	**78.56±0.94**	95.81±0.56	95.06±0.21	**94.53±0.12**	**94.21±0.47**	**79.64±1.56**
Multiple	Baseline	71.41±2.93	62.69±1.86	59.27±1.78	54.37±0.84	51.37±4.91	68.41±1.33	51.11±4.98	34.07±4.40	21.67±1.05	7.41±0.72
	SCE (2019)	84.45±2.45	84.27±0.68	82.15±1.99	83.39±0.74	79.21±3.09	76.29±2.55	63.81±5.51	48.05±2.13	25.30±5.51	22.67±3.49
	JoCoR (2020)	91.56±2.86	90.24±0.61	90.22±2.82	88.90±0.87	85.20±0.68	76.81±1.90	59.64±0.99	40.47±2.94	19.69±3.73	19.25±1.09
	RRL (2021)	75.70±2.23	73.67±2.90	65.84±4.92	59.83±5.75	56.09±3.30	72.82±1.97	56.41±4.66	39.41±1.44	23.42±2.29	11.38±1.23
	UNICON (2022)	93.59±1.79	93.85±0.78	92.49±2.69	90.70±1.29	87.17±2.91	88.61±0.78	86.63±0.90	84.29±1.05	80.27±2.70	31.56±3.86
	AGCE (2023)	93.11±1.25	93.65±0.66	92.92±0.60	87.78±3.57	83.97±4.30	78.30±1.50	62.95±2.40	52.22±2.73	43.58±1.03	26.35±0.93
	SPRL (2023)	84.91±2.30	75.59±3.10	68.26±4.93	61.21±2.64	52.26±4.78	73.89±8.60	59.54±7.23	33.31±1.28	21.71±1.12	7.15±1.01
	RML (2024)	85.99±7.48	79.97±7.28	77.47±9.08	66.83±7.21	34.46±6.12	**97.79±0.48**	84.40±3.74	69.26±7.73	45.58±5.16	33.22±0.88
	LSL (2024)	**97.63±0.76**	**96.19±0.58**	**95.69±1.08**	92.30±1.05	88.16±2.03	97.63±1.08	**96.93±0.82**	95.10±0.86	84.16±1.27	42.74±2.57
	MgCF (**Ours**)	96.96±0.97	95.88±0.86	95.81±0.46	**94.17±0.62**	**91.72±0.84**	97.31±0.57	96.40±0.70	**95.91±0.14**	**95.11±0.28**	**57.65±2.75**

TABLE s8: Comparison of effectiveness between MgCF and other methods for for learning with noisy labels. (The backbone, i.e. the encoder in Fig.2, is *MgNet*)

Datasets	Methods	Accuracy(%) under **asymmetric** noise with different rates					Accuracy(%) under **symmetric** noise with different rates				
		$\eta = 25\%$	$\eta = 30\%$	$\eta = 35\%$	$\eta = 40\%$	$\eta = 45\%$	$\eta = 50\%$	$\eta = 60\%$	$\eta = 70\%$	$\eta = 80\%$	$\eta = 90\%$
Damage	Baseline	83.01±0.67	81.65±0.48	79.31±0.86	74.19±0.99	57.86±1.71	74.17±1.65	69.69±2.03	62.83±1.68	40.75±2.68	13.36±2.82
	SCE (2019)	88.07±0.34	87.36±0.34	86.76±0.43	85.12±0.08	76.58±1.62	86.29±0.13	84.13±0.21	81.38±1.22	70.36±1.37	26.76±1.69
	JoCoR (2020)	91.69±0.31	91.06±0.60	90.13±0.13	88.16±0.19	80.83±1.47	89.62±1.30	87.57±1.36	82.78±2.42	71.41±0.53	26.13±5.35
	RRL (2021)	96.70±0.08	95.04±0.49	94.54±0.80	92.71±0.91	85.26±0.43	92.36±0.67	89.80±1.58	85.70±2.86	76.43±2.57	28.55±1.68
	UNICON (2022)	95.38±0.94	94.31±0.98	94.09±0.53	92.07±0.70	83.28±1.27	93.88±1.04	90.84±1.07	85.27±2.69	74.79±2.83	37.88±3.18
	AGCE (2023)	96.99±0.09	95.76±0.10	95.50±1.17	94.25±1.06	85.82±0.83	96.57±0.07	94.97±0.13	91.50±0.51	84.38±4.54	41.55±4.21
	SPRL (2023)	94.42±0.25	93.56±0.32	92.77±0.45	90.90±0.30	76.48±0.65	91.44±0.14	89.51±0.26	84.00±2.41	71.00±3.57	34.57±2.75
	RML (2024)	97.36±0.26	97.07±0.30	**96.89±0.07**	89.23±6.33	79.80±0.83	**97.48±0.65**	95.29±0.74	85.43±0.82	56.23±4.90	17.36±2.25
	LSL (2024)	**98.69±0.23**	**97.98±0.96**	96.83±0.94	91.01±0.90	83.94±0.27	97.38±0.53	95.24±0.82	91.58±0.92	82.35±2.10	69.37±3.49
	MgCF (**Ours**)	98.32±0.57	97.88±0.47	96.34±0.23	**94.50±0.65**	**86.82±1.55**	97.09±0.08	**95.84±0.40**	**92.20±0.24**	**90.40±0.35**	**76.69±0.69**
Multiple	Baseline	81.79±1.15	77.00±0.34	70.16±1.01	64.76±1.62	58.97±3.72	61.37±2.09	48.33±3.66	35.84±5.38	23.63±4.79	11.95±2.68
	SCE (2019)	90.13±0.23	87.15±0.30	86.89±0.14	82.30±0.32	75.94±1.08	80.87±2.27	76.67±1.37	66.10±0.76	39.16±5.14	14.37±2.17
	JoCoR (2020)	92.33±0.41	92.62±0.21	90.41±0.37	83.18±0.15	80.66±0.34	84.68±0.98	80.14±2.30	65.56±2.26	38.65±3.27	11.25±0.12
	RRL (2021)	95.69±0.51	94.67±0.51	90.14±0.47	82.18±0.22	71.39±0.64	87.35±2.03	81.72±2.85	63.06±4.66	35.94±5.33	13.53±1.77
	UNICON (2022)	96.84±0.73	95.33±0.36	94.98±0.87	86.17±0.90	83.75±1.27	87.20±0.82	83.02±1.61	67.63±2.46	39.28±4.15	13.86±0.55
	AGCE (2023)	93.29±0.34	93.03±0.24	93.13±0.19	86.31±0.33	74.22±0.45	90.64±0.43	88.42±1.17	80.87±4.48	46.96±4.82	15.24±2.55
	SPRL (2023)	95.09±0.50	93.86±0.83	93.18±0.48	82.42±0.14	77.39±1.90	85.91±0.86	77.54±0.74	65.19±7.00	30.17±7.97	14.65±2.22
	RML (2024)	96.04±0.85	95.27±0.09	**94.63±1.07**	83.07±2.51	74.14±1.36	**93.63±1.74**	77.61±2.73	55.96±4.85	34.10±3.02	19.17±0.97
	LSL (2024)	**97.05±0.38**	**96.45±0.75**	94.44±0.96	85.58±0.14	80.04±0.85	93.25±0.65	**90.99±1.73**	79.13±1.75	68.10±3.05	27.35±3.73
	MgCF (**Ours**)	96.59±1.17	95.74±0.52	94.54±0.53	**90.82±2.36**	**84.36±2.72**	93.09±1.35	90.24±1.75	**82.88±2.40**	**76.97±3.48**	**34.84±3.57**

TABLE s9: Detailed results on ablation of MgCF

Backbone	Datasets	Methods	Accuracy(%) under **asymmetric** noise with different rates					Accuracy(%) under **symmetric** noise with different rates				
			$\eta = 25\%$	$\eta = 30\%$	$\eta = 35\%$	$\eta = 40\%$	$\eta = 45\%$	$\eta = 50\%$	$\eta = 60\%$	$\eta = 70\%$	$\eta = 80\%$	$\eta = 90\%$
MSACNN	*Damage*	M1	92.32±0.74	91.68±0.11	90.21±0.04	85.70±1.37	71.71±1.43	94.54±0.38	94.19±0.06	90.68±0.16	86.31±1.10	64.26±1.41
		M2	93.50±0.32	92.90±0.28	93.48±0.12	79.60±1.00	54.63±1.68	94.75±0.10	93.14±0.28	89.91±0.14	73.90±1.08	40.89±2.76
		M3	94.36±0.72	94.38±0.28	92.99±0.51	81.54±0.36	66.22±0.91	95.77±0.63	94.94±0.34	92.81±0.30	89.12±0.64	67.16±3.56
		M4	94.86±0.03	94.71±0.16	93.06±0.18	89.23±0.72	74.83±3.17	95.75±0.14	94.44±0.17	92.63±0.23	79.46±0.73	56.45±3.46
	Multiple	M1	95.61±0.23	94.73±0.44	92.60±0.52	89.22±0.31	81.56±1.78	95.17±0.38	93.09±0.64	92.73±0.89	87.07±1.78	51.20±4.80
		M2	93.50±0.07	92.90±0.25	90.86±0.91	85.60±0.97	69.39±0.12	94.75±0.67	92.14±0.71	89.91±0.91	73.90±1.11	36.78±2.61
		M3	94.79±0.39	93.25±0.69	92.21±0.72	91.81±1.25	87.62±2.22	96.06±0.92	93.98±0.94	94.07±1.69	91.64±0.45	54.78±0.98
		M4	93.92±0.89	93.12±1.37	92.68±0.81	91.87±0.12	86.14±0.06	96.46±0.72	96.14±0.56	93.73±0.64	80.81±1.20	42.89±2.19
MgNet	*Damage*	M1	97.57±0.04	97.15±0.53	96.42±0.42	79.21±0.63	77.14±1.54	96.56±0.16	94.99±0.76	90.12±0.28	77.52±0.62	42.32±6.07
		M2	96.83±0.30	95.14±0.35	93.04±0.41	78.60±1.00	66.61±2.05	94.79±0.49	93.48±0.79	86.91±0.98	68.86±1.71	38.61±8.49
		M3	98.56±0.06	98.46±0.09	97.68±0.49	93.60±0.62	84.07±0.32	97.38±1.10	95.13±0.30	93.12±0.58	78.83±0.25	75.28±3.98
		M4	98.49±0.32	98.20±0.31	96.50±0.43	92.08±0.60	81.96±0.57	97.50±0.16	95.74±0.67	92.77±0.87	76.21±0.67	58.63±4.02
	Multiple	M1	95.91±0.44	94.22±0.53	93.55±0.32	88.83±0.47	80.34±0.71	91.66±1.32	84.59±1.95	70.84±2.86	59.03±7.40	24.38±2.75
		M2	96.81±0.11	93.14±0.35	90.21±0.53	85.53±0.61	73.65±2.75	90.21±0.49	75.91±0.79	57.05±3.75	48.58±6.57	18.61±7.34
		M3	96.14±0.60	95.47±0.56	93.66±0.16	85.95±0.72	83.93±1.90	92.44±0.23	93.78±1.91	81.24±4.44	74.05±6.00	31.05±3.89
		M4	96.83±0.28	95.11±0.02	94.25±0.38	91.07±0.50	77.06±0.48	92.58±0.54	89.09±0.74	77.09±2.37	65.70±5.98	27.86±3.97

s1.4 Supplementary experimention

In this section, we present two additional experiments to investigate the effects of training set size and data augmentation on overfitting in low-noise environments. These experiments aim to further explore the potential overfitting issues briefly mentioned in the manuscript and provide deeper insights into the factors that influence the performance of MgCF under different training conditions.

Our findings reveal that MgCF is highly sensitive to training set size. Once the training set reaches 60% or 70% of the total available samples, performance improves significantly, and beyond this point, increasing the training set size does not result in substantial performance gains. This suggests that MgCF can tolerate a reduction in training set size without significant overfitting, as long as the dataset is sufficiently large.

Furthermore, our exploration of data augmentation strategies indicates that several techniques can enhance model robustness, although not all are essential for achieving optimal performance. Among the five tested augmentation strategies (Cropping, Flip, Jitter, Noise, and Warping), jitter and noise were particularly effective in boosting model performance, while flip and warping showed more modest benefits. However, given that MgCF already demonstrates robust performance in the current setup, a fixed data augmentation pipeline may not be necessary for most fault diagnosis scenarios.

In conclusion, when a sufficiently large training dataset is available, MgCF is well-suited for fault diagnosis tasks. Appropriate data augmentation techniques can further improve its generalization capability. Detailed experimental results are provided in the following subsections.

s1.4.1 Case SI: Effect of training set size on MgCF performance

This experiment aims to analyze the impact of different training set sizes on the performance of MgCF. Specifically, we trained two diagnostic frameworks using varying proportions of available training samples (ranging from 10% to 90%, in 10% increments) across two datasets with different levels of noise, and evaluated their performance. The results are shown in Fig. s2.

We observed that MgCF exhibits a certain sensitivity to training set size. When the training set was reduced to less than 60% of its original size, the performance of both diagnostic models significantly dropped across all noise environments. However, as the training set size increased, model performance improved and stabilized. Once around 70% of the training data was used, performance approached its optimal level. Beyond this threshold, further increasing the training set size did not lead to substantial performance gains, suggesting that the training sample size was sufficient to capture the underlying patterns in the data.

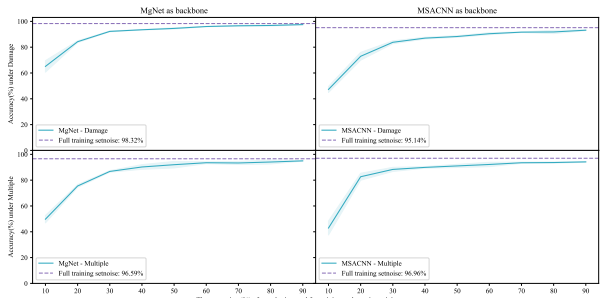
These results indicate that while MgCF can tolerate a reduction in training set size to some extent, smaller training sets lead to overfitting and loss of generalization, which supports our hypothesis that overfitting may occur in low-noise environments when the model becomes overly dependent on the available training data. In our experiment, we found that using approximately 70% of the training samples achieved performance comparable to that obtained with the full training set, highlighting that this sample size is sufficient for MgCF to generalize effectively in such scenarios.

s1.4.2 Case SII: Impact of data augmentation on the effectiveness of MgCF

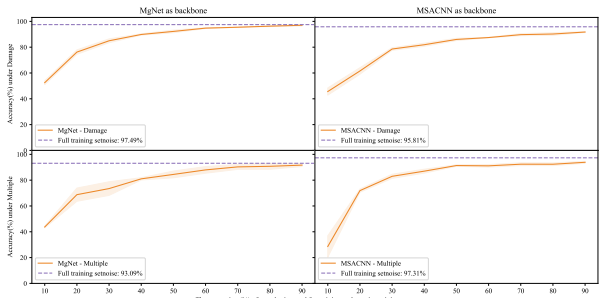
The second experiment explores the impact of data augmentation on the performance of MgCF. We tested five different time-series data augmentation strategies—Cropping, Flip, Jitter, Noise, and Warping—to evaluate how these techniques could help mitigate overfitting, see Fig. s3. Specifically, Warping enhances the data by applying nonlinear transformations along the time axis; Flip reverses the time dimension; Cropping performs random cropping on each sample, similar to random cropping in image augmentation; Jitter introduces random variations, with a maximum jitter of 4 data points; and Noise adds random Gaussian or Laplacian noise to the original data, targeting a signal-to-noise ratio of 5 dB.

Data augmentation generally improves MgCF performance by enhancing the model's robustness to small variations and noise in the input data. Among the five strategies, Jitter and Noise were particularly effective in improving the model's ability to generalize in noisy conditions, boosting performance in both training and testing phases. On the other hand, Cropping and Warping provided moderate improvements, but the performance gains were less significant compared to Jitter and Noise. Flip, while beneficial overall, had a relatively small impact on model performance when compared to the other augmentation strategies.

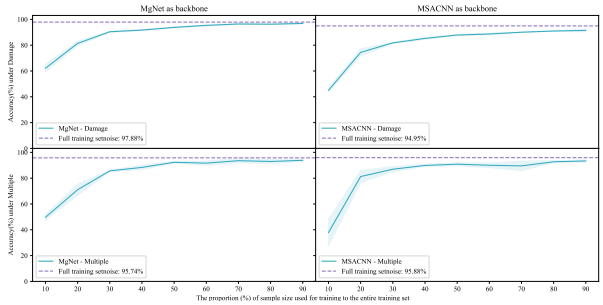
Despite these positive results, it is important to note that MgCF does not include any specific data augmentation strategies in its fixed preprocessing pipeline. This is because, with the current approach, the model's performance is already sufficiently robust, and additional augmentation strategies did not lead to substantial improvements under all noise conditions. Furthermore, some augmentation strategies, such as Flip and Warping, may introduce distortions that, while useful for certain models, could potentially harm MgCF's performance if applied indiscriminately. Therefore, the decision to forgo a fixed augmentation pipeline reflects the fact that **MgCF, under its current setup, already demonstrates strong performance** and may not require further augmentation for most fault diagnosis scenarios.



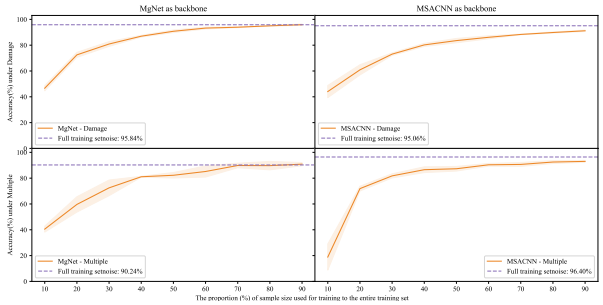
(a) Under 25% of asymmetric noise



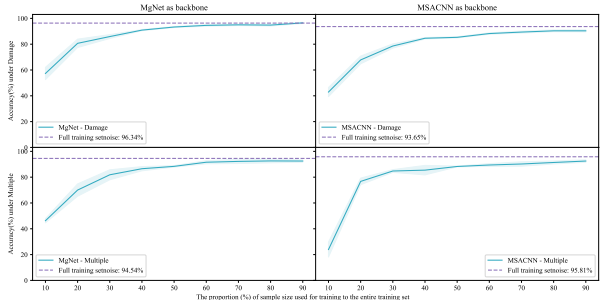
(b) Under 50% of symmetric noise



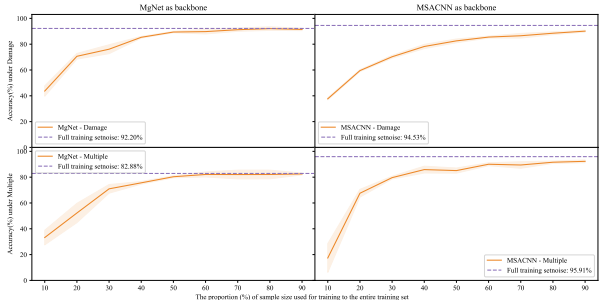
(c) Under 30% of asymmetric noise



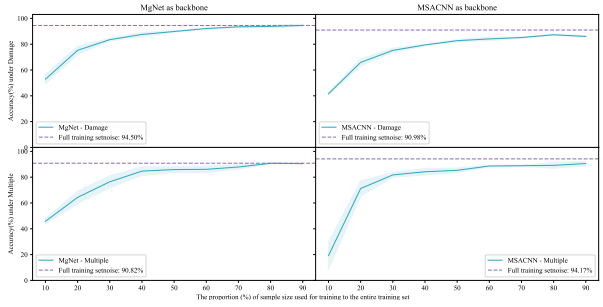
(d) Under 60% of symmetric noise



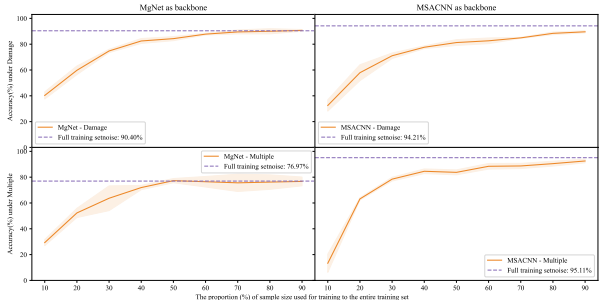
(e) Under 35% of asymmetric noise



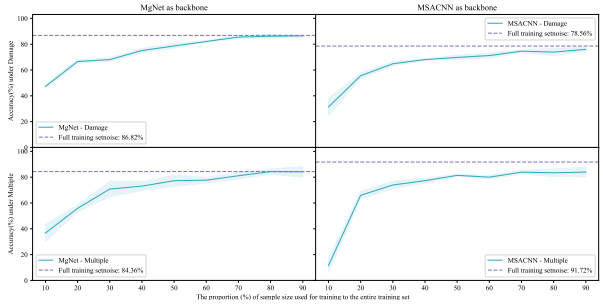
(f) Under 70% of symmetric noise



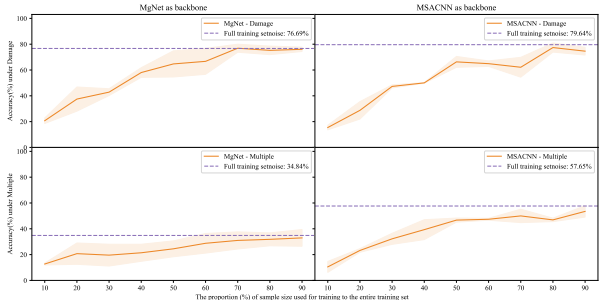
(g) Under 40% of asymmetric noise



(h) Under 80% of symmetric noise

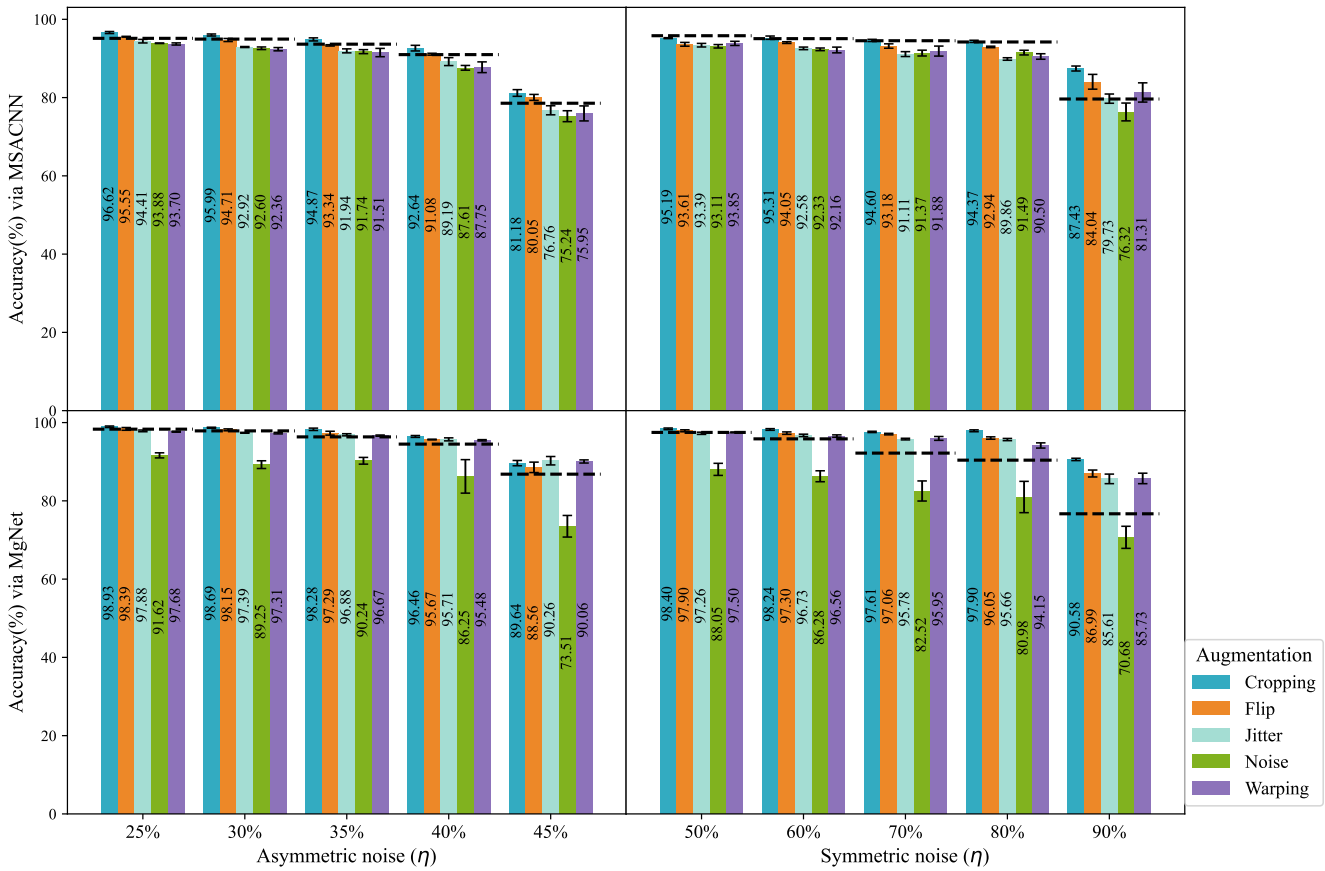


(i) Under 45% of asymmetric noise

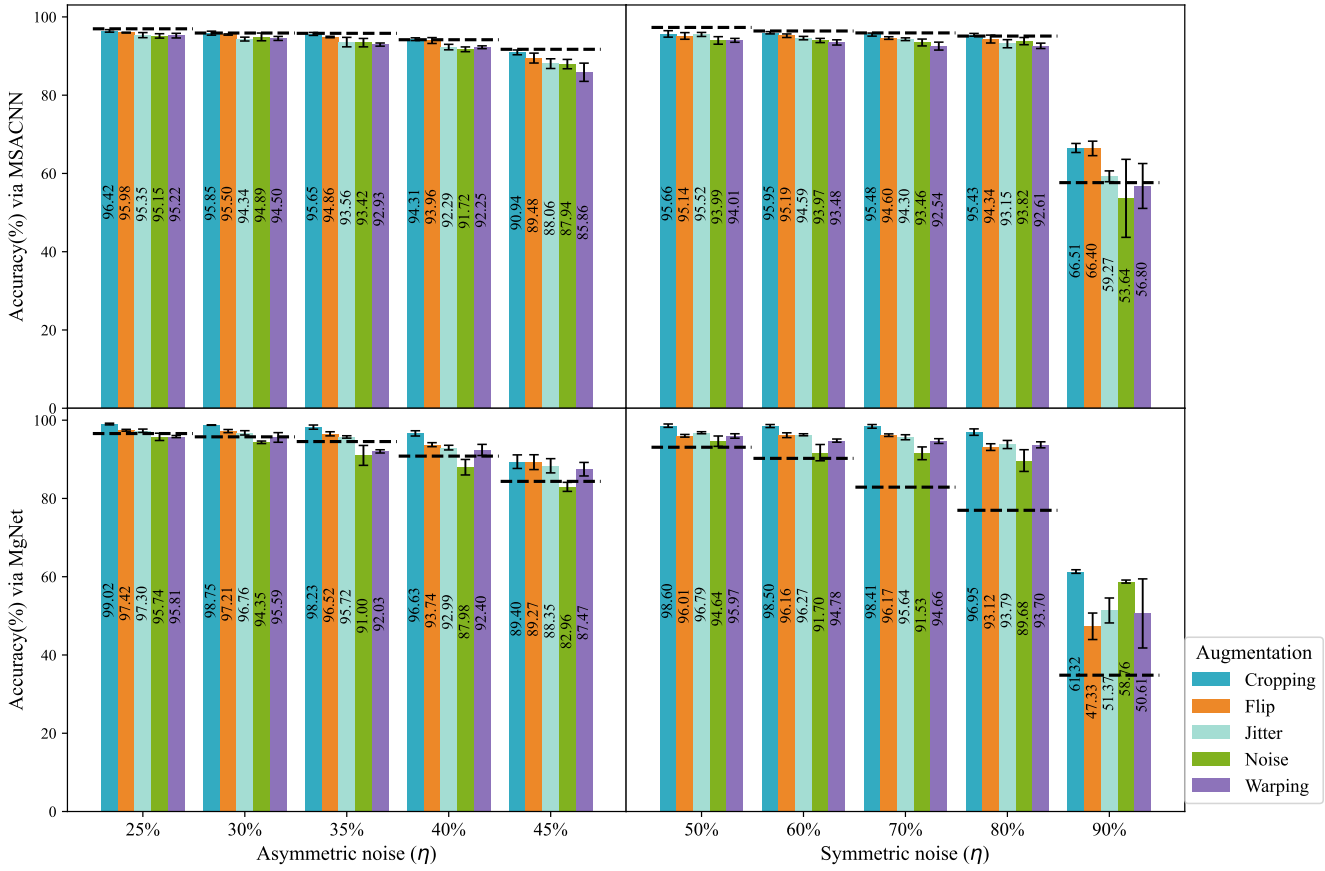


(j) Under 90% of symmetric noise

Fig. s2: The impact of training with different proportions of the original training set samples on the performance of MgCF. The dashed line represents the average accuracy achieved after training with the full training set data.



(a) Results on *Damage* dataset



(b) Results on *Multiple* dataset

Fig. s3: The impact of different data augmentation strategies on MgCF.

s1.5 Visualization analysis

s1.5.1 Estimated noise intensity

In this subsection, we first visualize the consistency between the estimated noise intensity ($\tilde{\eta}$) and the actual noise intensity (η) during the training process, which aims to further validate the necessity of the constructed confidence correction term, as this consistency can approximately reflect the accuracy of the membership generated via cluster fusion. We randomly selected data from three training sessions out of multiple repeated experiments and plotted the estimated noise intensity for each epoch during training (Fig. s4). As shown in Fig. s4, the repeated visual results demonstrate very high consistency, indirectly reflecting the stability of MgCF's effectiveness. This high reproducibility reduces the likelihood of random initialization affecting the results, thus confirming the robustness of MgCF.

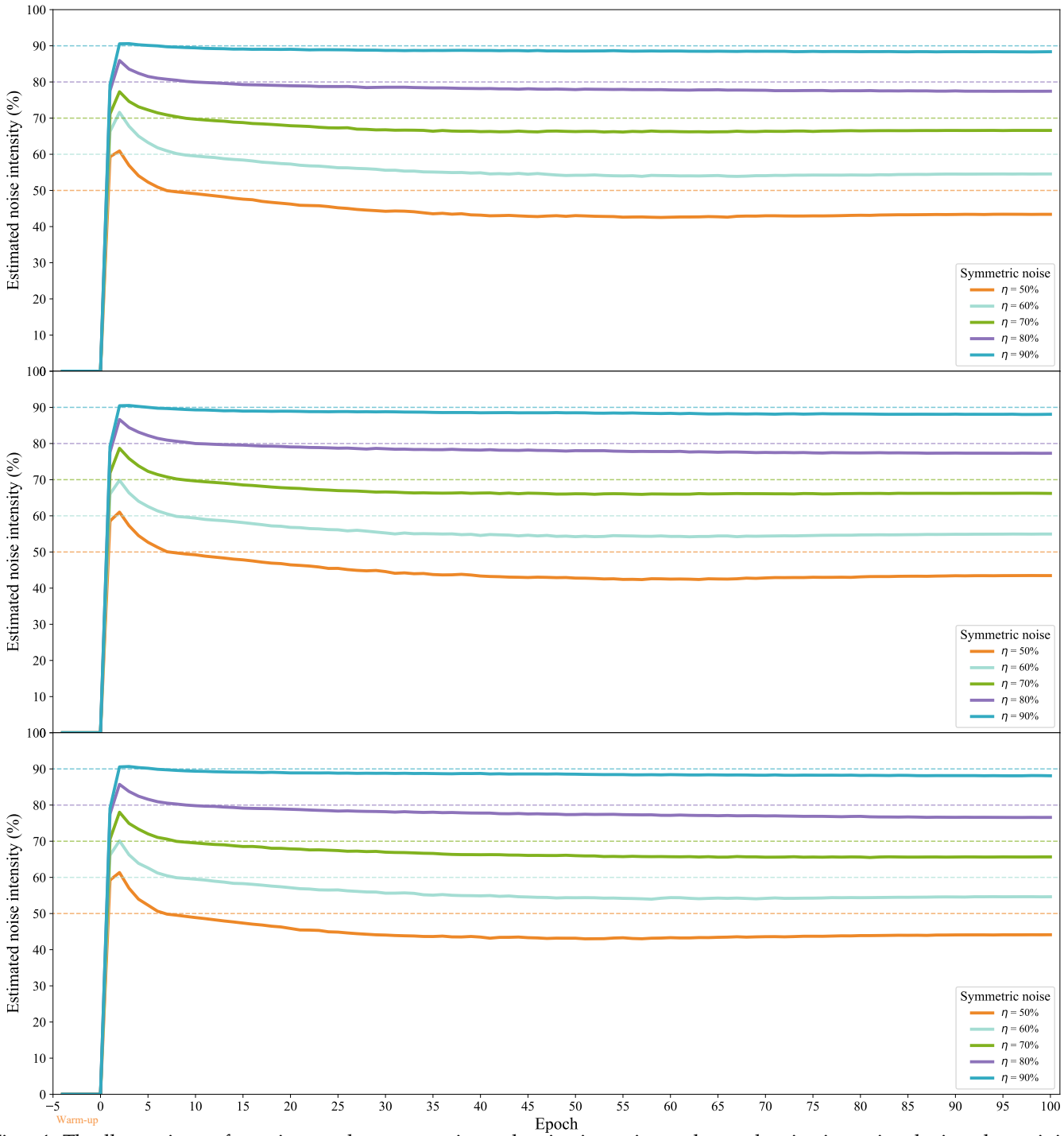


Fig. s4: The llustrations of consistency between estimated noise intensity and actual noise intensity during the training process. Where the dashed line represents the reference line for the actual added symmetric noise intensity. Negative epochs indicate the estimated noise intensity recorded during the warm-up phase.

In high-noise environments (symmetric noise intensity greater than 70%), the error between the estimated noise intensity

and the actual noise intensity remains within 3%, an acceptable range, which indicates that the category membership function constructed through cluster fusion can effectively reflect the trustworthiness of the observed labels. Across all noise intensities, the estimated noise intensity converges to a certain range as epochs increase and always remains slightly lower than the actual noise intensity, which suggests that achieving more accurate observed label confidence requires appropriate adjustments to the category membership function. This observation further supports the necessity of the confidence correction term of Eq. (8).

In low-noise environments, particularly in the mid-to-late stages of training, the estimated noise intensity is significantly lower than the actual noise intensity. We attribute this to the model possibly converging to a well-generalized local optimum. Since the category membership function based on cluster fusion relies on information from cluster center samples, there may be some degree of overfitting in the later stages, excessively reducing the confidence in noisy labels and thereby diminishing the noise resistance advantage of multi-granularity labels. This phenomenon explains why MgCF shows significant advantages in high-noise environments but performs similarly to other state-of-the-art methods (e.g. RML and LSL, etc.) in low-noise environments, as observed in Sec. IV-B. Thus, like other state-of-the-art methods, MgCF may not fully overcome the challenge of cumulative error amplification during the sself-guided learning process.

s1.5.2 Distribution of latent features

To further validate the hypothesis that the significant error between the estimated and actual noise intensities in low-noise environments is due to overfitting in the later stages of training, we used t-SNE to visualize the feature space under different noise conditions (Fig. s5). The visualizations include the distribution of the test set in the feature space of the baseline model, as well as the distribution of the test and training sets in the feature space of the MgCF-trained model.

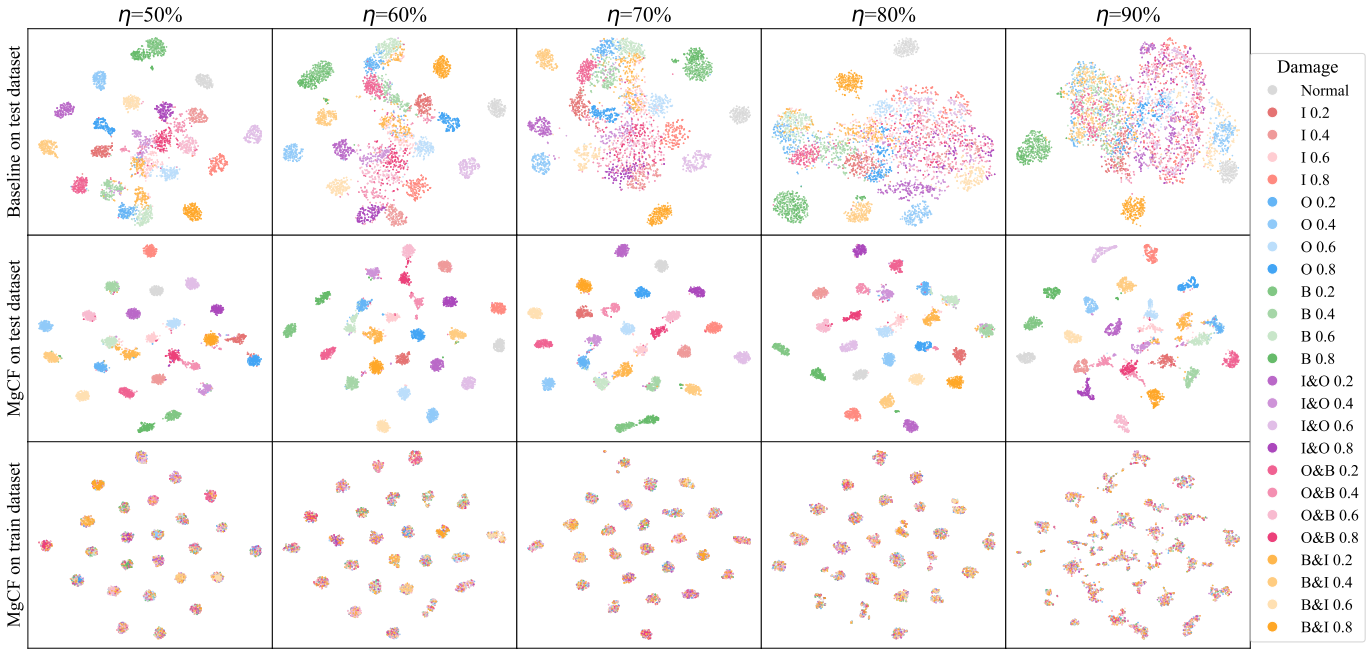


Fig. s5: The visualization of the dataset with different noise levels in the latent feature space of the trained via t-SNE.

The baseline's feature space visualization for the test set shows a dispersed distribution at different noise levels, which indicates that the baseline struggles to maintain data consistency and coherence, especially as noise levels increase. The dispersed nature suggests a lack of robustness to noise, leading to poor generalization. In contrast, the feature space of model trained by MgCF for the test set shows more structured and coherent clusters. At lower noise levels, the clusters are more distinct and separated, indicating better generalization and noise robustness. However, as noise levels increase, the clusters begin to overlap slightly but still maintain better separation than the baseline model. This structured clustering validates MgCF's effectiveness in handling noisy labels and enhancing model robustness. For the training set, the feature space of MgCF shows even more distinct and tightly clustered groups, which indicates that the model learns well from the training data even in the presence of noise. However, this tight clustering also suggests the possibility of overfitting, especially in low-noise environments, i.e. model may focus excessively on the noise-free aspects of the data, reducing its ability to generalize when exposed to new, unseen data.

The observed overfitting in low-noise environments explains why the estimated noise intensity is significantly lower than the actual noise intensity in the mid-to-late stages of training. The model converges to a local optimum that generalizes well on the training data but struggles with new data. The category membership function based on cluster fusion, determined by the cluster centroid samples, contributes to this overfitting, leading to decreased confidence in noisy labels and reduced noise resistance, which explains why MgCF shows significant advantages in high-noise environments but only performs comparably to state-of-the-art methods in low-noise environments, without demonstrating a remarkable improvement.

s1.6 Computational complexity analysis of MgCF

MgCF's complexity primarily arises from three components: feature embedding and clustering, class attribution calculation, and label fusion with model updating.

Feature embedding is performed using deep neural networks such as MgNet or MSACNN, which map input samples into a high-dimensional feature space. This is followed by clustering based on sample similarity, such as cosine similarity. The complexity of computing the similarity matrix for a dataset with N samples and d-dimensional features is $O(N^2 \cdot d)$. The computational demands of feature extraction depend on the network architecture; for example, MgNet and MSACNN have forward pass complexities of $O(5.49\text{MB})$ and $O(267.63\text{MB})$, respectively. Class membership degrees for cluster centers are then calculated using the Dezert-Smarandache theory, which combines similarity and label information. Assuming g represents clustering granularity, this step has a complexity of $O(N \cdot g)$. Finally, multi-granularity labels are generated by weighting observed and pseudo-labels, followed by standard supervised training. The training complexity aligns with that of traditional models, depending on the network and dataset size.

In comparison with other strategies, MgCF offers a distinct balance between complexity and flexibility. Noise transition matrix-based approaches, for instance, have a complexity of $O(K^2)$ (where K is the number of classes), which is lower than MgCF's clustering and class attribution steps. However, MgCF's multi-granularity framework provides greater robustness and adaptability. Sample selection methods, which often rely on loss-based filtering, operate with a linear complexity $O(N)$, but they may sacrifice robustness by excluding valuable information from the training process. Data augmentation and pseudo-labeling strategies, with complexities proportional to $O(N \cdot \text{augmentation iterations})$, introduce additional overhead, while MgCF avoids this by directly leveraging multi-granularity information.

Experimental results demonstrate that MgCF achieves significant performance improvements across various noise levels while maintaining efficient execution, even with computationally demanding models like MSACNN. This highlights its effective balance between computational complexity and performance.

To make a long story short, MgCF is well-suited for medium-sized datasets and applications requiring high noise tolerance and robustness. For large-scale datasets, optimization techniques such as sparse matrix approximations or efficient similarity calculation methods may further enhance scalability. Practical implementations could benefit from strategies like hierarchical clustering or KD-tree-based indexing to reduce clustering complexity, as well as GPU acceleration to expedite feature extraction and similarity computations.

S2 PROOFS

s2.1 Supplementary analysis and discussion of *Assumption 1*

Assumption 1 posits that for any two samples $x^{(i)}$ and $x^{(j)}$ in the dataset $\mathcal{D}$, if the function $f(\Theta_b, \Theta_c; \cdot)$ generalizes well, then the similarity $Sim(z^{(i)}, z^{(j)})$ between their feature representations $z^{(i)}$ and $z^{(j)}$ should be proportional to the probability that they share the same label, i.e., $p(y^{(i)} = y^{(j)})$. This assumption suggests that samples with high similarity in feature space are more likely to belong to the same class, forming the basis for MgCF's label consistency approach across clusters.

To empirically validate this assumption, we conducted a statistical analysis of the relationship between feature similarity and label consistency. The results in **Fig. 3** indicate that as feature similarity increases, the probability of label consistency also rises significantly, which provides quantitative support for **Assumption 1**, affirming that samples with high similarity in feature space are indeed more likely to share the same label.

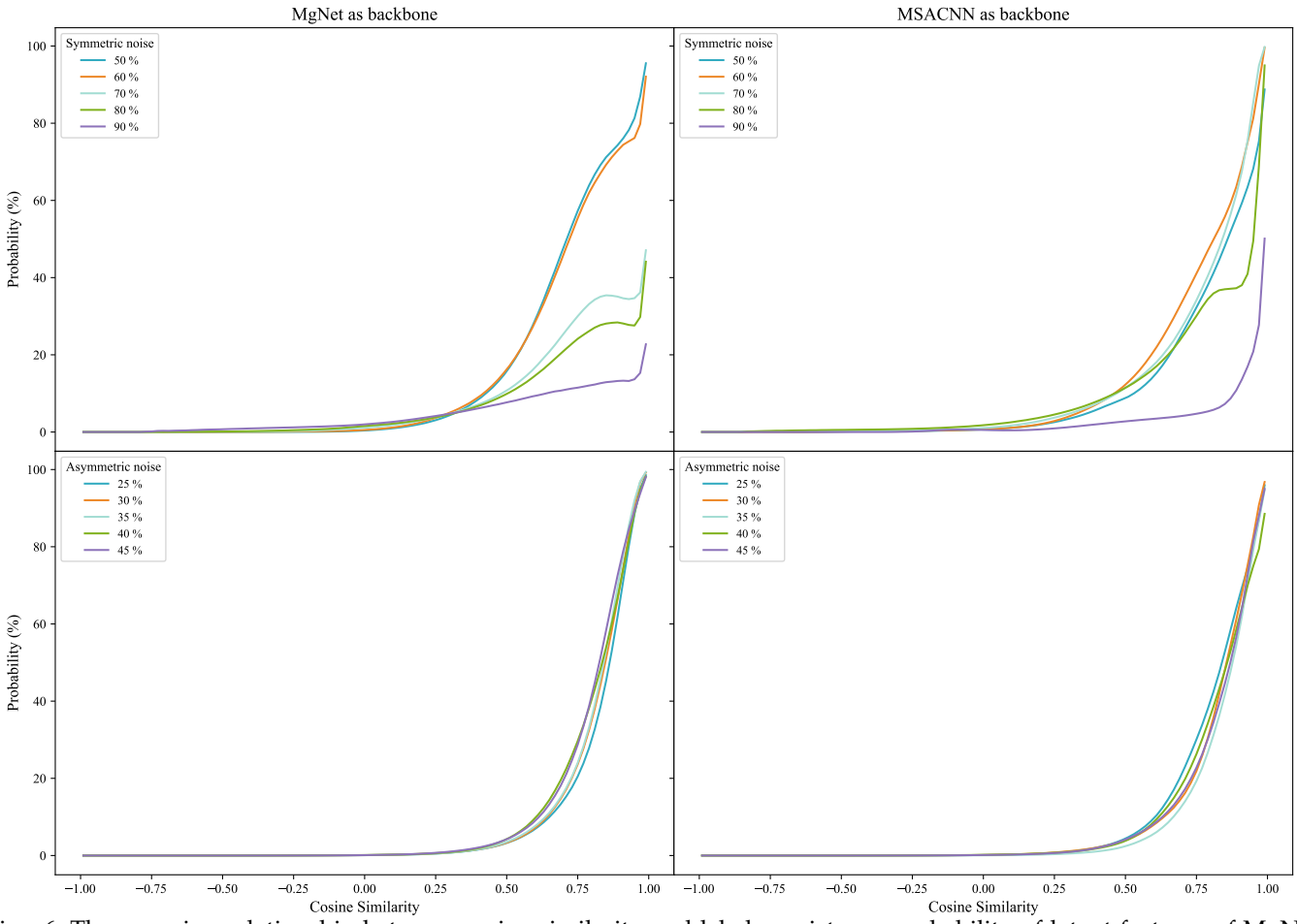


Fig. s6: The mapping relationship between cosine similarity and label consistency probability of latent features of MgNet and MSACNN under different noise environments on the *Damage* dataset.

Further details are shown in Fig. s6 and Fig. s7, where the consistency between feature similarity and label consistency probability is examined under different noise conditions and for various model architectures (i.e., MgNet and MSACNN). Both models reveal a consistent trend: as feature similarity increases, the probability of label consistency also increases, supporting **Assumption 1** across different noise levels. This relationship holds for both symmetric and asymmetric noise, indicating the robustness of **Assumption 1** under moderate noise conditions.

However, as noise levels reach extreme intensities (e.g., 90% of symmetric noise), the relationship between feature similarity and label consistency becomes weaker, as shown in Fig. s7. For the MSACNN model at a 90% noise level, the probability of label consistency does not increase as significantly with feature similarity, suggesting that **Assumption 1** becomes a weaker assumption in highly overlapping clusters or extreme noise environments. To gain further insight, we conducted a t-SNE visualization analysis of the latent feature space under various noise levels, as presented in Sec. s1.5.2 and Fig. s5.

The t-SNE visualizations reveal distinct differences in the feature distributions for both the baseline and MgCF-trained models. For the baseline model, the test set's feature distribution appears dispersed and inconsistent across different noise levels, highlighting its struggle to maintain data coherence and generalization under high noise. In contrast, the MgCF-trained model demonstrates more structured and coherent clusters in the feature space, even at high noise levels. At lower noise intensities, the clusters are well-separated, indicating strong generalization and noise robustness. As noise

levels increase, clusters start to overlap but still maintain better separation than those of the baseline model, underscoring MgCF's ability to handle noisy labels effectively.

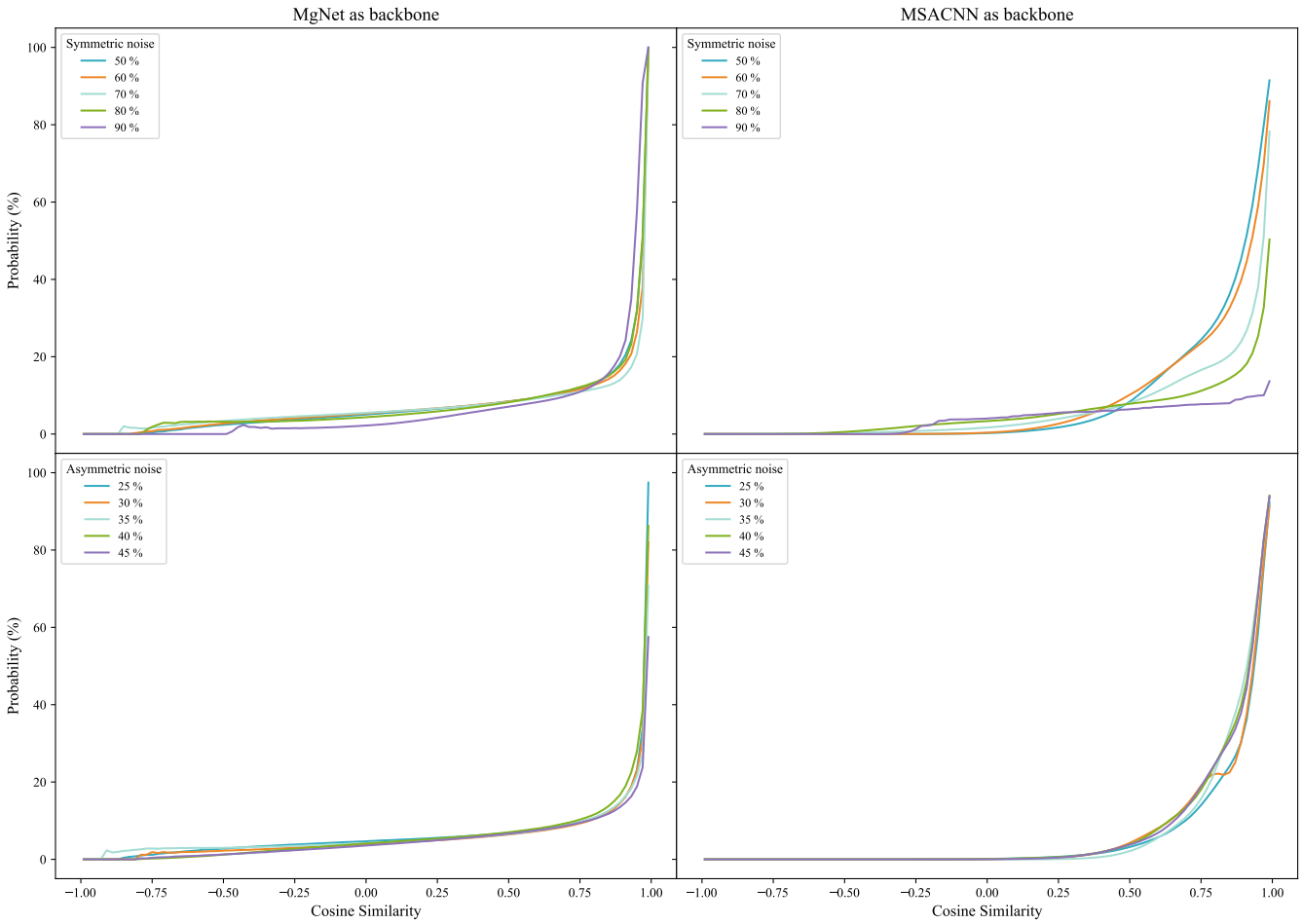


Fig. s7: The mapping relationship between cosine similarity and label consistency probability of latent features of MgNet and MSACNN under different noise environments on the *Multiple* dataset.

The t-SNE results suggest that while Assumption 1 holds under moderate noise, in extreme noise conditions where clusters overlap significantly, this assumption becomes weaker. To address this limitation, **MgCF** introduces a multi-granularity labeling mechanism that adapts based on the confidence of the pseudo-labels. Specifically, for high-confidence pseudo-labels that match the original label, MgCF generates fine-grained labels to ensure precise correction. In contrast, for low-confidence pseudo-labels that differ from the original label, MgCF combines the pseudo-label with the original observation to create a coarse-grained label that includes an uncertainty factor. This approach enables MgCF to incorporate hesitation in cases where Assumption 1 does not fully hold, allowing the model to prioritize reliable corrections and effectively mitigate confirmation bias.

In summary, our experiments validate **Assumption 1** in general scenarios with moderate noise, demonstrating a strong correlation between feature similarity and label consistency. However, in high-noise environments where clusters overlap heavily, MgCF's multi-granularity label mechanism effectively addresses the weakened assumption by adapting label granularity based on pseudo-label confidence. The t-SNE visualizations and probability analyses collectively demonstrate MgCF's robustness in handling noisy labels and its resilience to challenges introduced by high noise levels, highlighting the adaptability and effectiveness of the multi-granularity approach.

s2.2 Proof for noisy tolerance

Theorem 1: (***Noisy tolerance of MgCF***) *: For a K-classification task, when there exists a globally optimal model f_g and the model f is trained using cross entropy as the loss function $\ell(p,y) = -\sum_{i=1}^{K} y_i \log(p_i)$, if the noise intensity η in the dataset satisfies $\eta < 1 - \frac{1}{K}$ when the noise type is symmetric (uniform), or satisfies $0 < \eta_{kl} < 1 - \eta$ with $\sum_{k \neq l} \eta_{kl} = \eta$ when the noise type is asymmetric (class-dependency), then the fused labels $\widetilde{\mathcal{Y}}$ have noise tolerance.*

Proof. For symmetric noise:

Let $\mathcal{D}_\eta$ represents a dataset with symmetric noise labels of intensity η, and there is a probability distribution $P(\hat{y} = k \mid y = k) = 1 - \eta$ for $\forall k \in [1, K]$, then the risk of model f on $\mathcal{D}_\eta = \{(\mathcal{X}, \widetilde{\mathcal{Y}})\}$ for the fused labels $\widetilde{\mathcal{Y}}$ is:

$$\begin{aligned} R_\eta(f) &= \mathbb{E}_{\mathcal{X},\widetilde{\mathcal{Y}}}[\ell(f(x),\tilde{y})] \\ &= \mathbb{E}_{\mathcal{D}_\eta}[\ell(f(x), \nu \times \hat{y} + (1-\nu) \times \overline{y})] \\ &= \mathbb{E}_{\mathcal{D}_\eta}[\nu \times \ell(f(x),\hat{y})] + \mathbb{E}_{\mathcal{D}_\eta}[(1-\nu) \times \ell(f(x),\overline{y})] \end{aligned} \tag{s1}$$

Wherein:

$$\begin{aligned} \mathbb{E}_{\mathcal{D}_\eta}[\ell(f(x),\hat{y})] &= \mathbb{E}_{\mathcal{X},\widehat{\mathcal{Y}}}[\ell(f(x),\hat{y})] \\ &= \mathbb{E}_{\mathcal{X},\mathcal{Y}}[(1-\eta) \times \ell(f(x),y) + \tfrac{\eta}{K-1} \times \textstyle\sum_{k \neq y} \ell(f(x),k)] \\ &= (1-\eta) \times R(f) + \tfrac{\eta}{K-1} \times \mathbb{E}_{\mathcal{X},\mathcal{Y}}[\textstyle\sum_{k=1}^{K} \ell(f(x),k) - \ell(f(x),y)] \\ &= (1-\eta) \times R(f) + \tfrac{\eta}{K-1} \times \mathbb{E}_{\mathcal{X},\mathcal{Y}}[\textstyle\sum_{k=1}^{K} \ell(f(x),k)] - \tfrac{\eta}{K-1} \times R(f) \\ &= (1 - \tfrac{\eta}{K-1}K) \times R(f) + \tfrac{\eta}{K-1} \times \mathbb{E}_{\mathcal{X},\mathcal{Y}}[\textstyle\sum_{k=1}^{K}(-\log f_k(x))] \\ &= (1 - \tfrac{\eta}{K-1}K) \times R(f) + \tfrac{\eta K}{K-1} \times R_{LS,\frac{K-1}{K}}(f) \end{aligned} \tag{s2}$$

Then, by introducing the conditions of the conditions of **Assumption 1** and Eq. (s2) into Eq. (s1), there are:

$$\begin{aligned} R_\eta(f) &= \eta \times \mathbb{E}_{\mathcal{X},\widehat{\mathcal{Y}}}[\ell(f(x),\hat{y})] + (1-\eta) \times \mathbb{E}_{\mathcal{X},\overline{\mathcal{Y}}}[\ell(f(x),\overline{y})] \\ &= \eta \times [(1 - \tfrac{\eta}{K-1}K) \times R(f) + \tfrac{\eta}{K(K-1)}] + (1-\eta) \times \mathbb{E}_{\mathcal{D}}[\ell(f(x), y^{LS,\eta})] \\ &= \eta((1 - \tfrac{\eta}{K-1}K) \times R(f) + \tfrac{\eta K}{K-1} \times R^{LS,\frac{K-1}{K}}(f)) + (1-\eta) \times R^{LS,\eta}(f) \end{aligned} \tag{s3}$$

Thus,

$$\begin{aligned} R_\eta(f_g) - R_\eta(f) = \quad & \eta(1 - \tfrac{\eta K}{K-1}) \times (R(f_g) - R(f)) \\ & + \eta \tfrac{\eta K}{K-1} \times (R^{LS,\frac{K-1}{K}}(f_g) - R^{LS,\frac{K-1}{K}}(f)) \\ & + (1-\eta) \times (R^{LS,\eta}(f_g) - R^{LS,\eta}(f)) \end{aligned} \tag{s4}$$

Where, let $y^{LS,r} = \frac{(1-r)(K-1)-r}{K-1} \times y + \frac{r}{K-1} \times \mathbf{1}$ denote the label smoothing operation with smoothing factor r, then:

$$\begin{aligned} R^{LS,r}(f) &= \mathbb{E}_{\mathcal{D}}[\ell(f(x), y_{LS,r})] \\ &= \mathbb{E}_{\mathcal{X},\mathcal{Y}}[\ell(f(x), \tfrac{(1-r)(K-1)-r}{K-1} \times y + \tfrac{r}{K-1} \times \mathbf{1})] \\ &= \mathbb{E}_{\mathcal{X},\mathcal{Y}}[\ell(f(x), \tfrac{(1-r)(K-1)-r}{K-1} \times y)] + \mathbb{E}_{\mathcal{X},\mathcal{Y}}[\tfrac{r}{K-1} \times \textstyle\sum_{k=1}^{K} \ell(f(x),k)] \\ &= \tfrac{(1-r)(K-1)-r}{K-1} \times R(f) + \tfrac{rK}{K-1} \times R^{LS,\frac{K-1}{K}}(f) \end{aligned} \tag{s5}$$

After substituting Eq. (s5) into Eq. (s4):

$$\begin{aligned} R_\eta(f_g) - R_\eta(f) = \quad & (\eta(1 - \tfrac{\eta K}{K-1}) + (1-\eta) \times \tfrac{(K - \eta K - 1)}{K-1}) \times (R(f_g) - R(f)) \\ & + \tfrac{\eta K}{K-1} \times (R^{LS,\frac{K-1}{K}}(f_g) - R^{LS,\frac{K-1}{K}}(f)) \end{aligned} \tag{s6}$$

Given that $K \geq 2$ and $0 \leq \eta < 1 - \frac{1}{K}$ in **Theorem 1**, we consider:

$$\left.\begin{array}{ll} 0 \leq & \eta < 1 - \frac{1}{K} \\ 1 < & K - \eta K \\ 0 < & 1 - \frac{\eta K}{K-1} \end{array}\right\} \Rightarrow \eta(1 - \frac{\eta K}{K-1}) > 0 \tag{s7}$$

Meanwhile, according to **Definition 1**, for $\forall$f $0 \leq R(f_g) \leq R(f)$ is established and Inequality $R^{LS,\frac{K-1}{K}}(f_g) - R^{LS,\frac{K-1}{K}}(f) \propto R(f_g) - R(f) \leq 0$ always holds, thereupon obtaining:

$$R_\eta(f_g) - R_\eta(f) \leq 0 \tag{s8}$$

Eq. (s8) proves that f_g is also a risk $R_\eta(f)$ The global minimization of fused label $\tilde{y}$ has noisy tolerance, that is, the fused multi-granularity labels $\widetilde{\mathcal{Y}}$ is **robust to symmetric noisy labels on** $\mathcal{D}_\eta$.

Proof. For asymmetric noise, i.e. class-dependent noise:

Let $\mathcal{D}_\eta$ represents a dataset with asymmetric noise labels of intensity η and $\sum_{k\neq y}\eta_{yk}=\eta$, and there is the probability distributions $P(\hat{y}=k\mid y=k)=1-\eta$ and $P(\hat{y}=j\neq i=y)=\eta_{ij}$ for $\forall i,j,k\in[1,K]$, then following the symmetric case, the risk of classifier f on $\mathcal{D}_\eta$ for the fused labels $\widetilde{\mathcal{Y}}$ still satisfies Eq. (s1).
Wherein:

$$\begin{aligned}\mathbb{E}_{\mathcal{D}_\eta}[\ell(f(x),\hat{y})] &= \mathbb{E}_{\mathcal{X},\widehat{\mathcal{Y}}}[\ell(f(x),\hat{y})]\\ &= \mathbb{E}_{\mathcal{X},\mathcal{Y}}[(1-\eta)\times\ell(f(x),y)+\textstyle\sum_{k\neq y}\eta_{yk}\times\ell(f(x),k)]\\ &= (1-\eta)\times\mathbb{E}_{\mathcal{X},\mathcal{Y}}[\textstyle\sum_{k=1}^{K}\times\ell(f(x),k)-\sum_{k\neq y}\ell(f(x),k)]+\mathbb{E}_{\mathcal{X},\mathcal{Y}}[\sum_{k\neq y}\eta_{yk}\times\ell(f(x),k)]\\ &= (1-\eta)\times\mathbb{E}_{\mathcal{X},\mathcal{Y}}[K\times\textstyle\sum_{k=1}^{K}\times\ell(f(x),\frac{k}{K})]+\mathbb{E}_{\mathcal{X},\mathcal{Y}}[\sum_{k\neq y}(\eta+\eta_{yk}-1)\times\ell(f(x),k)]\\ &= (1-\eta)\times\mathbb{E}_{\mathcal{X},\mathcal{Y}}[\textstyle\sum_{k=1}^{K}\ell(f(x),K\times\frac{k}{K})]+\frac{K+K\eta+1}{K-1}\mathbb{E}_{\mathcal{X},\mathcal{Y}}[\sum_{k\neq y}(K-1)\times\ell(f(x),\frac{k}{K-1})]\\ &= K(1-\eta)\times R^{LS,\frac{K-1}{K}}(f)+(K+K\eta+1)\times R^{LS,1}(f)\end{aligned}\tag{s9}$$

Thus, here we have,

$$\begin{aligned}R_\eta(f) &= \eta\times\mathbb{E}_{\mathcal{D}_\eta}[\ell(f(x),\hat{y})]+(1-\eta)\times\mathbb{E}_{\mathcal{D}_\eta}[\ell(f(x),\overline{y})]\\ &= \eta\times[K(1-\eta)\times R^{LS,\frac{K-1}{K}}(f)+(K+K\eta+1)\times R^{LS,1}(f)]\\ &\quad+(1-\eta)\times(1-\eta)\times R^{LS,\eta}(f)\\ &= K(1-\eta)\eta\times R^{LS,\frac{K-1}{K}}(f)+(K+K\eta+1)\eta\times R^{LS,1}(f)+(1-\eta)^2\times R^{LS,\eta}(f)\end{aligned}\tag{s10}$$

$$\Downarrow$$

$$\begin{aligned}R_\eta(f_g)-R_\eta(f) &= K(1-\eta)\eta\times(R^{LS,\frac{K-1}{K}}(f_g)-R^{LS,\frac{K-1}{K}}(f))\\ &\quad+(K+K\eta+1)\eta\times(R^{LS,1}(f_g)-R^{LS,1}(f))\\ &\quad+(1-\eta)^2\times(R^{LS,\eta}(f_g)-R^{LS,\eta}(f))\end{aligned}\tag{s11}$$

As f_g with the minimizer $R(f)$ ($0\leq R(f_g)\leq R(f)$) and $\forall r\in[0,1]\Rightarrow 0\leq R^{LS,r}(f_g)\leq R^{LS,r}(f)$. So, according to Eq. (s11) and $0<\eta_{yk}<1-\eta_y$, it is evident that:

$$R_\eta(f_g)-R_\eta(f)\leq 0\tag{s12}$$

Ergo, for this case of asymmetric noise, **the fused multi-granularity labels $\widetilde{\mathcal{Y}}$ have the noisy tolerance** for $\mathcal{D}_\eta$ with asymmetric noise.

S3 REFERENCE CODE FOR IMPLEMENTING

s3.1 Pseudo code

Alg. s1 The core code of MgCF

```
def MgCF(net, loader, eta, categories, InitialGranularity = 100):

   net.eval().to(device)

   # Determine the granularity of feature clusters based on estimated dataset noise intensity
   num_g = max( int( eta * InitialGranularity ), 10 )

   samples, targets, latents = [], [], []
   for sample, target in loader:
      y = F.one_hot(target, categories)
      with torch.no_grad():
         z = F.normalize( net.backbone(x) )

      samples.append( x )
      targets.append( y )
      latents.append( z )
   samples, targets, latents = torch.cat(samples, 0), torch.cat(targets, 0), torch.cat(latents, 0)

   # Calculate similarity between features
   similarity = torch.einsum('bl, gl -> bg', latents, latents ).exp().fill_diagonal(0)
   # Obtain the belief mass of each sample within all feature clusters
   belief, index = torch.topk( similarity, num_g, dim=1 )

   # Obtaining membership functions via cluster fusion
   membership = (sub_targets[index] * belief.unsqueeze(dim = 2)).sum(1) / belief.sum(1, keepdim = True)

   purity, pseudo = membership.max(dim = 1)
   pseudo = F.one_hot(pseudo, categories)
   mu = torch.einsum('bc, bc -> b', membership, targets )

   certainty = mu * mu/purity
   uncertainty = 1 - certainty

   # Obtain multi-granularity labels by fusing pseudo labels and original observation labels
   MgTargets = torch.einsum('bc,b->bc', targets, certainty) + torch.einsum('bc,b->bc', pseudo, uncertainty)
   # Update the estimation of noise intensity on the dataset
   eta = 1 - purity.mean().item()

   MgDatasets = TensorDataset( samples, MgTargets )

   return MgDatasets, eta
```

s3.2 The complete training code of MgCF, as a reference implementation way

```
# Initialization settings

import os
os.environ["CUDA_VISIBLE_DEVICES"] = "7"

import numpy as np
import time
import argparse
from scipy.io import savemat
from tqdm.notebook import tqdm

import torch
import torch.nn as nn
from torch.nn import functional as F
from torch.utils.data import DataLoader, TensorDataset, random_split
from torch.cuda.amp import GradScaler, autocast

torch.backends.cudnn.enable =True
torch.backends.cudnn.benchmark = True

'''
The version of python is  3.10.14
The version of torch is  2.5.1
The version of numpy is  1.26.4
```

```
The version of argparse is  1.1
'''

import warnings
warnings.filterwarnings("ignore")

parser = argparse.ArgumentParser(description = "Global HyperParameter")
parser.add_argument('-f', type=str, default = "Read additional parameters")

parser.add_argument('--LogSavedir', default = 'Case I' )
parser.add_argument('--savedir', default = 'Exper. 001 ( MgCF via MgNet )',
                    help = "Experimental result saving address sub-file name")
parser.add_argument('--Logdir', default = os.path.join('../Saved Results'))

parser.add_argument('--SampleLength', default = 2**11 )
parser.add_argument('--DataRoot', default = os.path.join('/Datasets/Bearings with Noisy labels'))
parser.add_argument('--Datasets', default = [ 'Multiple', 'Damage' ] )
parser.add_argument('--NL', default = 'Noisy labels' )
parser.add_argument('--NoiseType', default = {
    'Symmetric':[ 90, 80, 70, 60, 50 ], 'Asymmetric':[ 45, 40, 35, 30, 25 ] } )

parser.add_argument('--Dims', default = 2**6 )
parser.add_argument('--Granularity', default = 100 )

parser.add_argument('--Epochs', default = [ 5, 100 ] )
parser.add_argument('--InitialLearningRate', default = 1e-3 * 3.0)
parser.add_argument('--BatchSize', default = 2**9 )

parser.add_argument('--device', default = torch.device("cuda" if torch.cuda.is_available() else "cpu"))
parser.add_argument('--num_workers', default = 0)

Args = parser.parse_args()
if not os.path.exists(Args.Logdir):
    os.mkdir(Args.Logdir)

Args.ParentLogdir = os.path.join(Args.Logdir, Args.LogSavedir)
if not os.path.exists(Args.ParentLogdir):
    os.mkdir(Args.ParentLogdir)
Args.logdir = os.path.join(Args.ParentLogdir, Args.savedir)
if not os.path.exists(Args.logdir):
    os.mkdir(Args.logdir)

def GetTensorDataset( training, dataname, NoiseType, intensity ):

    '''
    training: 'tes' or 'tra'
    '''

    npy_x = training + '_samples.npy'
    npy_y = training + '_targets.npy'

    path_x = os.path.join(os.path.join(Args.DataRoot, 'Clean'), dataname)

    if training == 'tra':
        path_y = os.path.join(os.path.join( os.path.join(
            os.path.join(Args.DataRoot, Args.NL), NoiseType), dataname ), str(intensity))
    else:
        path_y = os.path.join(os.path.join(Args.DataRoot, 'Clean'), dataname)

    samples, targets = [], []
    for domain in os.listdir(path_x):

        data_x = os.path.join( path_x, domain )
        sample = np.load(os.path.join( data_x, npy_x ))

        data_y = os.path.join(path_y, domain)
        target = np.load(os.path.join(data_y, npy_y))

        samples.append( torch.as_tensor(sample, dtype = torch.float32) )
        targets.append( torch.as_tensor(target, dtype = torch.int64) )

    dataset = TensorDataset( torch.cat(samples), torch.cat(targets) )

    if training == 'tes':
        loader = DataLoader(dataset, batch_size = Args.BatchSize,
                            num_workers = Args.num_workers, shuffle = False)
        return loader
```

```
    else:
        total_size = len(dataset)
        val_size = int(total_size * 0.2)
        tra_size = total_size - val_size
        tra_dataset, val_dataset = random_split(dataset, [tra_size, val_size])
        tra_loader = DataLoader(tra_dataset, batch_size = Args.BatchSize,
                                num_workers = Args.num_workers, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size = Args.BatchSize,
                                num_workers = Args.num_workers, shuffle=True)

        return tra_loader, val_loader, target.max() + 1

# Model architecture
'''
Diagnostic Framework

@article{deng2023mgnet,
  title={MgNet: A fault diagnosis approach for multi-bearing system
  based on auxiliary bearing and multi-granularity information fusion},
  author={Deng, Jin and Liu, Han and Fang, Hairui and Shao, Siyu
  and Wang, Dong and Hou, Yimin and Chen, Dongsheng and Tang, Mingcong},
  journal={Mechanical Systems and Signal Processing},
  volume={193},
  pages={110253},
  year={2023},
  publisher={Elsevier}
}
'''
class Linear(nn.Module):
    def __init__(self, in_features, out_features,
                 norm = None, act = None,
                 zeros_init = False, with_bias = True ):
        super().__init__()

        self.Linear = nn.Linear(in_features, out_features)

        if zeros_init:
            nn.init.zeros_(self.Linear.weight)
        else:
            nn.init.kaiming_uniform_(self.Linear.weight)

        if with_bias:
            nn.init.zeros_(self.Linear.bias)

        self.norm = norm
        self.act = act

    def forward(self, x):

        x = self.Linear(x)

        if self.norm is not None:
            x = self.norm(x)
        if self.act is not None:
            x = self.act(x)

        return x

class Conv1D(nn.Module):
    def __init__(self, in_channels, out_channel,
                 kernel = 5, stride = 1, norm = None, act = None):
        super().__init__()

        self.conv = nn.Conv1d(in_channels, out_channel, kernel, stride, padding = int((kernel-1)/2))

        nn.init.kaiming_uniform_(self.conv.weight)
        nn.init.zeros_(self.conv.bias)

        self.norm = norm
        self.act = act

    def forward(self, x):

        x = self.conv(x)

        if self.norm is not None:
            x = self.norm(x)
```

```
        if self.act is not None:
            x = self.act(x)

        return x

class Add(nn.Module):
    def __init__(self, number = 2):
        super().__init__()
        self.weights = nn.Parameter(torch.ones(number, dtype=torch.float32), requires_grad=True)

    def forward(self, x):
        w = F.relu(self.weights)
        return x[0]*w[0] + x[1]*w[1]

class ACON(nn.Module):
    def __init__(self):
        super().__init__()
        self. = nn.Parameter(torch.ones(2, dtype=torch.float32), requires_grad=True)

    def forward(self, x):
         = F.relu(self.)
        x = [0] * x * torch.sigmoid(x * [1])
        return x

class Mg(nn.Module):
    def __init__(self, d_in, d_out):
        super().__init__()

        self.convs = nn.ModuleList([
            nn.Sequential(
                Conv1D( d_in, d_out//4, kernel = 2**(k+2)+1, stride = 2,
                       norm = nn.BatchNorm1d(d_out//4), act = ACON() ),
                Conv1D( d_out//4, d_out//4, kernel = 2**(k+2)+1, stride = 2,
                       norm = nn.BatchNorm1d(d_out//4), act = ACON() )
                ) for k in range(4)
        ])

        self.add = Add()

        self.idx = nn.Sequential( Conv1D(
            d_in, d_out, kernel = 1, stride = 1,
            norm = nn.BatchNorm1d(d_out), act = ACON() ),
                                 nn.MaxPool1d(4, 4) )

        self.point = Conv1D( d_out, d_out, kernel = 1, stride = 1,
                            norm = nn.BatchNorm1d(d_out), act = ACON() )

    def forward(self, x):

        i = self.idx(x)

        xes = []
        for conv in self.convs:
            xes.append(conv(x))
        x = torch.cat(xes, dim = 1)

        x = self.point(x)

        x = self.add([x, i])

        return x

class MgNet_backbone(nn.Module):
    def __init__( self, dims, Stages = [4, 16, 64, 128] ):
        super().__init__()

        '''
        Stages: In the original MgNet experiment, the Stages was set to [4, 16, 64, 128]
        and the default single channel data input, which can be modified according to actual needs.
        '''

        self.stages = nn.Sequential(
            Mg(1, Stages[0]), Mg(Stages[0], Stages[1]),
            Mg(Stages[1], Stages[2]), Mg(Stages[2], Stages[3])
            )

        self.weights = nn.Parameter(torch.as_tensor([1, 0], dtype=torch.float32), requires_grad=True)
```

```
        self.projector = Linear( Stages[3], dims )

    def forward(self, x):

        weights = F.relu( self.weights )
        z = self.stages(x)

        return self.projector( z.mean(2) * weights[0] + z.max(2)[0] * weights[1] )

class Get_Model(nn.Module):
    def __init__(self, categories, dims = Args.Dims ):
        super().__init__()

        self.backbone =  MgNet_backbone( dims )
        self.predictor = Linear( dims, categories, zeros_init = True )

    def forward(self, x):

        z = self.backbone(x)
        prediction = self.predictor( z )

        return prediction

# Model training

def smooth_one_hot(true_labels, categories, smoothing = 0.3):
    confidence = smoothing/(categories-1)
    true_labels = F.one_hot(true_labels, categories)
    with torch.no_grad():
        smooth_label = true_labels * (1.0 - smoothing - confidence) + confidence
    return smooth_label

def GetAccuracy( model, loader, categories ):

    model.eval().to(Args.device)
    criterion = nn.CrossEntropyLoss()
    criterion.to(Args.device).eval()

    prefetcher = iter(loader)

    count, correct, losses = 0, 0, []

    for _ in range(len(loader)):
        sample, target = next(prefetcher)
        count += target.shape[0]
        target = target.to(Args.device)

        y = F.one_hot(target, categories).float()
        x = sample.to(Args.device)

        with torch.no_grad():
            prediction = model(x)

        loss = criterion( prediction, y )
        losses.append( loss.item()  )
        correct += torch.eq(prediction.argmax(dim=1), target).sum().float().item()

    return correct/count * 100, np.mean(losses)

def GradientUpdate( model, loader, optimizer, scheduler, categories ):
    '''
    Supervised learning process (Warm-up)
    '''
    model.train()
    model.to(Args.device)
    criterion = nn.CrossEntropyLoss()
    criterion.eval()
    criterion.to(Args.device)

    count, correct, losses = 0, 0, []
    scaler = GradScaler()
    prefetcher = iter(loader)

    for _ in range(len(loader)):

        sample, target = next(prefetcher)
        count += target.shape[0]
```

```
        y = smooth_one_hot( target.to(Args.device), categories )
        x = sample.to(Args.device)

        with autocast():
            prediction = model( x )
            loss = criterion(prediction, y)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        losses.append( loss.item()  )

        optimizer.zero_grad()
        scheduler.step()

        correct += torch.eq( prediction.argmax(dim=1), y.argmax(dim=1) ).sum().float().item()

    return correct/count * 100, np.mean(losses)

def CorrectionLoader( model, loader, categories, intensity ):

    model.eval()
    model.to(Args.device)

    samples, targets, latents = [], [], []
    for sample, target in loader:
        x = sample.to(Args.device)
        y = F.one_hot( target.to(Args.device), num_classes=categories).float()

        with torch.no_grad():
            z = F.normalize( model.backbone(x) )

        samples.append( x )
        targets.append( y )
        latents.append( z )

    samples = torch.cat(samples, 0).to('cpu')
    targets = torch.cat(targets, 0)
    latents = torch.cat(latents, 0)

    granularity = max( int( intensity * Args.Granularity ), 10 )

    datasets, Intensity = [], []
    similarity = torch.einsum('bl, gl -> bg', latents, latents ).exp()
    similarity.fill_diagonal_(0)
    weight, index = torch.topk( similarity, granularity, dim=1 )

    SgClabels = (targets[index] * weight.unsqueeze(dim = 2)).sum(1) / weight.sum(1, keepdim = True)

    purity, pseudo = SgClabels.max(dim = 1)
    pseudo = F.one_hot( pseudo, num_classes=categories).float()
    intensity = 1 - purity.mean().item()

    belief = torch.einsum('bc, bc -> b', SgClabels, targets )
    certainty = belief * ( belief/purity )
    uncertainty = 1 - certainty

    Mg_targets = torch.einsum('bc, b -> bc', targets, certainty) + torch.einsum('bc, b -> bc', pseudo, uncertainty)

    loader = DataLoader(
        TensorDataset( samples, Mg_targets.to('cpu') ),
        batch_size = Args.BatchSize, num_workers = Args.num_workers, pin_memory=True, shuffle = True )

    return loader, intensity

def GradientUpdateViaCorrectedLabels( model, loader, optimizer, scheduler ):

    model.train()
    model.to(Args.device)
    criterion = nn.CrossEntropyLoss()
    criterion.eval()
    criterion.to(Args.device)

    count, correct, losses = 0, 0, []
    scaler = GradScaler()
```

```
    for sample, target in loader:
        count += target.shape[0]

        y = target.to(Args.device)
        x = sample.to(Args.device)

        with autocast():
            p = model(x)
            loss = criterion(p, y)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
        losses.append( loss.item()  )

        optimizer.zero_grad()
        scheduler.step()

        correct += torch.eq( p.argmax(dim=1), y.argmax(dim=1) ).sum().float().item()

    return correct/count * 100, np.mean(losses)

def GetModelWithWeights( save_root, tra_loader, val_loader, categories ):

    training_net = os.path.join( Args.logdir, 'Training net.pt' )
    training_log = os.path.join( save_root, 'Training logs.mat' )

    Trained_net = os.path.join( save_root, 'Trained net.pt' )
    if not os.path.exists(training_log):
        model = Get_Model( categories )
        model.to(Args.device)

        Acc, ValAcc, Loss, ValLoss, Intensity, Max_acc = [], [], [], [], [], 0.
        intensity = 0.

        for e in range(len(Args.Epochs)):

            epoch = Args.Epochs[e]
            LearningRate = Args.InitialLearningRate
            optimizer = torch.optim.AdamW( model.parameters(), LearningRate )
            scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
                optimizer, T_max = epoch * len(tra_loader) + 1 )

            pbar = tqdm( range(epoch) )
            for _ in pbar:
                if e==0:
                    acc, loss = GradientUpdate( model, tra_loader, optimizer, scheduler, categories )
                else:
                    Mg_Loader, intensity = CorrectionLoader( model, tra_loader, categories, intensity )
                    acc, loss = GradientUpdateViaCorrectedLabels( model, Mg_Loader, optimizer, scheduler )

                val, val_loss = GetAccuracy( model, val_loader, categories )

                if val>Max_acc:
                    Max_acc = val
                    torch.save(model.state_dict(), training_net)

                pbar.set_description(
                    r'{} times training: with {:.2f}% of val acc, {:.2f}% of acc,\
                    {:.3f} of loss, {:.3f} of val loss {:.3f} of intensity.'.format(
                    e + 1, val, acc,  loss, val_loss, intensity * 100
                )
                                        )

                Acc.append(acc)
                Loss.append(loss)

                ValAcc.append(val)
                ValLoss.append(val_loss)

                Intensity.append(intensity * 100)

            model.load_state_dict( torch.load(training_net) )

        log_results = {}

        log_results['Accuracy'] = Acc
```

```
        log_results['ValAccuracy'] = ValAcc
        log_results['Loss'] = Loss
        log_results['ValLoss'] = ValLoss
        log_results['Intensity'] = Intensity

        savemat( training_log, log_results )
        torch.save(model.state_dict(), Trained_net)

    else:
        model = Get_Model( categories )
        model.load_state_dict( torch.load(Trained_net) )
        model.to(Args.device)

    return model

def Training( training_times = 10 ):

    test_results = os.path.join( Args.logdir, 'Test results.txt')

    if not os.path.exists(test_results):
        Header = '{}\t{}\t{}\t{}\t{}\t{}\t\n'.format(
            'Dataset', 'Noise type', 'Noise intensity', 'Times', 'Accuracy(%)', 'Loss' )
        with open(test_results, 'a+') as log:
            log.write(Header)

    for dataname in Args.Datasets:

        for nt in Args.NoiseType.keys():
            print()
            print('==========================================================================')
            for intensity in Args.NoiseType[nt]:
                print(nt, ' noisy         ', intensity, '%')

                for times in range(training_times):
                    times += 1

                    f = open(test_results, encoding='gbk')
                    Results=[]
                    for line in f:
                        Results.append(line.strip().split('\t'))

                    IfResult = [
                        result for result in Results
                        if result[0] == dataname
                        and result[1] == nt
                        and result[2] == str(intensity)
                        and result[3] == str(times)
                    ]

                    if len(IfResult) > 0:
                        for results in IfResult:
                            print( r'On {} dataset with {} noise of {}% {}-times:\
                            accuracy of {:.3f}% with loss of {:.4f}.'.format(
                                results[0], results[1], results[2], results[3],
                                float(results[4]), float(results[5])
                                )
                            )
                    else:
                        save_root = os.path.join( Args.logdir, dataname )
                        if not os.path.exists(save_root):
                            os.mkdir(save_root)

                        save_root = os.path.join( save_root, nt )
                        if not os.path.exists(save_root):
                            os.mkdir(save_root)
                        save_root = os.path.join( save_root, str(intensity) )
                        if not os.path.exists(save_root):
                            os.mkdir(save_root)

                        save_root = os.path.join( save_root, str(times) + ' times' )
                        if not os.path.exists(save_root):
                            os.mkdir(save_root)

                        tra_loader, val_loader, categories = GetTensorDataset( 'tra', dataname, nt, intensity )
                        tes_loader = GetTensorDataset( 'tes', dataname, nt, intensity )

                        model = GetModelWithWeights( save_root, tra_loader, val_loader, categories )
```

```
                        acc, loss = GetAccuracy( model, tes_loader, categories )

                        results = '{}\t{}\t{}\t{}\t{}\t{}\t\n'.format(
                            dataname, nt, intensity, times, acc, loss )
                        with open(test_results, 'a+') as log:
                            log.write(results)

                        results = results.split('\t')
                        print(r'On {} dataset with {} noise of {}% {}-times:\
                        accuracy of {:.3f}% with loss of {:.4f}.'.format(
                            results[0], results[1], results[2], results[3],
                            float(results[4]), float(results[5]) )
                            )

                print()

# Display experimental results

Training(  )
```